

Domain 3 — Performance Optimization, Querying & Transformation

Complete Beginner's Master Guide | COF-C03 2026

Technical + Layman Definitions | All Subdomains | Edge Cases | Hands-On SQL | 20 Questions Per Topic | Full Answer Key

How to use this guide

1. Read ☒ **Layman Explanation** first — build the mental model
2. Read ☒ **Technical Explanation** — precise mechanics, every edge case, exam traps
3. Run every **Hands-On Query** block in your Snowflake free trial
4. Attempt all 20 questions per topic **WITHOUT** peeking at answers
5. Check the annotated **Answer Key** at the very end

Exam facts: Domain 3 (Performance Optimization, Querying & Transformation) = 21% of the COF-C03 exam. It is the second-largest domain. The four subdomains are: 3.1 Query Profile, 3.2 Virtual Warehouse Configurations, 3.3 Warehouse Performance Tools, 3.4 Query Optimization Techniques.^{[1][2]}

SUBDOMAIN 3.1 — The Query Profile

☒ Layman's Explanation

Imagine you asked your team to build a piece of furniture. An hour later you come back and it's still not done. The **Query Profile** is the CCTV footage of that whole hour — it shows you exactly where everyone was stuck:

- Was one person carrying all the heavy boxes alone? (One node doing all the work)
- Did someone fill the floor with so many pieces that they had to move some to the hallway? (Data spilling to disk)
- Was nobody sure which box to open first? (Full table scan — no pruning)
- Was everyone waiting in line to use the one saw? (Warehouse queuing)

Snowflake's **Query Profile** in Snowsight is an interactive tree diagram of every step your query took, how long each step took, and exactly where bottlenecks occurred.^[^3]

☒ Technical Explanation

How to Access the Query Profile^[^3]

1. In Snowsight: Click Activity (left sidebar) → Query History → click any Query ID → click the "Query Profile" tab
2. Alternatively: Execute a query → at the bottom of the worksheet, click the Query ID link

The Query Profile is available for queries run in the last 14 days (Snowsight Query History retention).^[^3]

The Execution Plan DAG

The Query Profile displays a **Directed Acyclic Graph (DAG)** — a tree of operator nodes. Data flows from leaf nodes (table scans) up through processing nodes to the final result node at the top.

Key Operator Node Types:^{[4][1]}^[^3]

Node	What It Does	What to Look For
TableScan	Reads data from a Snowflake table	Partitions Scanned vs. Partitions Total (pruning efficiency); Bytes Scanned
Filter	Applies WHERE conditions after a scan	High row input with low row output = good filtering
Aggregate	Performs GROUP BY, COUNT, SUM, AVG	Can be expensive; look at input row count
Join	Combines two tables (Hash Join is Snowflake's primary join type)	Row count explosion = Exploding Join
Sort	Applies ORDER BY	Can trigger disk spilling if sort buffer overflows
Exchange	Redistributes data across virtual warehouse nodes	BROADCAST: small table sent to all nodes; PARTITION: data shuffled by join/group key
ExternalScan	Reads from an external table	Always slower than TableScan — data is in cloud storage
QUERY RESULT REUSE	Entire query served from Result Cache	No warehouse compute used; returns in milliseconds
METADATA-BASED RESULT	Query answered from Cloud Services metadata	No warehouse needed; e.g., COUNT(*), MIN(), MAX()
WithReference / WithClause	A CTE (WITH clause) reference or reuse	CTEs are materialized once in Snowflake

The Most Important Metrics to Read in Query Profile^{[4][3]}

Metric	Location	What It Means
Partitions Scanned	TableScan node	Number of micro-partitions actually read
Partitions Total	TableScan node	Total micro-partitions in the table
Bytes Scanned	TableScan node	Volume of data read from storage
Bytes Spilled to Local Storage	Overall stats	Data overflowed from memory → spilled to warehouse SSD
Bytes Spilled to Remote Storage	Overall stats	Data overflowed from SSD → spilled to cloud storage (VERY bad)
Time in Queue (Blocked Time)	Top-level query stats	Time waiting for a warehouse slot
Rows Produced	Each node	Output row count — explosions at Join nodes = problem
Percentage of Query Time	Each node	Which node is the biggest bottleneck

The 5 Performance Killers — Exam Favorites^{[5][6][7][4][^3]}

1. Inefficient Pruning (Full Table Scan)

- **Symptom:** `Partitions Scanned ≈ Partitions Total` in the TableScan node
- **Why it happens:** No clustering key on the filtered column; WHERE clause doesn't align with how data is physically stored
- **Fix:** Add a clustering key on the column used in WHERE filters; ensure WHERE clauses use indexed/clustered columns

2. Bytes Spilled to Remote Storage

- **Symptom:** `Bytes Spilled to Remote Storage > 0` in query stats
- **Why it happens:** Intermediate results exceed the warehouse's available memory AND SSD — overflow goes to cloud storage
- **Severity levels:** Local spill (minor performance hit) < Remote spill (severe: 10-100x slower)
- **Fix:** Scale UP the warehouse (more memory per node); reduce query complexity; add aggregation earlier

3. Exploding Joins (Join Explosion)

- **Symptom:** Row count OUT of a Join node is dramatically larger than row count IN (e.g., 1M rows in → 50B rows out)
- **Why it happens:** Missing or incorrect join key; unintentional Cartesian product; many-to-many relationship without deduplication

- **Fix:** Review join conditions; ensure proper join keys; deduplicate source before join; add WHERE filters
- Snowflake uses Hash Joins primarily — join explosion still possible if join condition is non-selective^[7]

4. Query Queuing (High Transaction Blocked Time)

- **Symptom:** High Time in Queue at the top of the query timeline
- **Why it happens:** Too many concurrent queries competing for the same warehouse slots
- **Fix:** Scale OUT (enable multi-cluster warehouse); increase MAX_CLUSTER_COUNT; separate workloads into dedicated warehouses

5. Large Scans with No Filters

- **Symptom:** Bytes Scanned = full table size; entire table is being read
- **Why it happens:** SELECT without a WHERE clause; missing or ignored clustering key
- **Fix:** Add WHERE clause; use clustering keys; use LIMIT for exploratory queries

Query Insights (Snowsight Built-In Flags)^{[5][3]}

Snowflake's Snowsight automatically flags known problem patterns in the Query Profile:

- INEFFICIENT_JOIN_CONDITION — Complex join condition evaluated after the join (post-join filter, not pre-filter)
- EXPLODING_JOIN — Join output significantly exceeds input sizes
- CARTESIAN_PRODUCT — No join condition or non-equi join producing a cross product
- INEFFICIENT_PRUNING — Partitions Scanned $\approx$ Partitions Total

Reading the Query Profile — Key Workflow^{[4][3]}

1. Look at "Most Expensive Nodes" — which operator consumed the most time?
2. Check the TableScan node — Partitions Scanned vs. Total (pruning ratio)
3. Check for spilling — any bytes spilled to local or remote storage?
4. Check Join nodes — did row counts explode?
5. Check Sort nodes — did they spill?
6. Check top-level query stats — was any time spent in queue?

☒ Hands-On Queries: Query Profile Exploration

```
-- =====
-- SETUP: Create tables for Query Profile demonstrations
-- =====

USE ROLE SYSADMIN;
CREATE DATABASE IF NOT EXISTS PERF_DEMO;
USE DATABASE PERF_DEMO;
CREATE SCHEMA IF NOT EXISTS QUERY_PROFILE_LAB;
USE SCHEMA QUERY_PROFILE_LAB;
USE WAREHOUSE learn_wh;

-- Create a fact table with 500,000 rows
CREATE OR REPLACE TABLE sales_fact (
    sale_id      INTEGER,
    sale_date    DATE,
    region       VARCHAR(30),
    product_id   INTEGER,
    customer_id  INTEGER,
    amount       NUMBER(12,2),
    quantity     INTEGER
);

INSERT INTO sales_fact
SELECT
    SEQ4()                                AS sale_id,
    DATEADD('day', -MOD(SEQ4(), 730), CURRENT_DATE()) AS sale_date,
    CASE MOD(SEQ4(), 4) WHEN 0 THEN 'North' WHEN 1 THEN 'South'
        WHEN 2 THEN 'East' ELSE 'West' END      AS region,
    MOD(SEQ4(), 1000) + 1                      AS product_id,
    MOD(SEQ4(), 50000) + 1                     AS customer_id,
    ROUND(UNIFORM(5.0, 5000.0, RANDOM(42)), 2) AS amount,
    UNIFORM(1, 100, RANDOM(42))                AS quantity
FROM TABLE(GENERATOR(ROWCOUNT => 500000));

-- Create a dimension table
CREATE OR REPLACE TABLE product_dim (
    product_id   INTEGER,
    product_name VARCHAR(100),
    category     VARCHAR(50),
    unit_price   NUMBER(10,2)
);

INSERT INTO product_dim
SELECT
    SEQ4() + 1                                AS product_id,
    'Product_' || (SEQ4() + 1)                AS product_name,
    CASE MOD(SEQ4(), 5)
        WHEN 0 THEN 'Electronics'
        WHEN 1 THEN 'Clothing'
        WHEN 2 THEN 'Food'
        WHEN 3 THEN 'Tools'
        ELSE 'Books'
    END                                         AS category,
    ROUND(UNIFORM(5.0, 500.0, RANDOM()), 2) AS unit_price
FROM TABLE(GENERATOR(ROWCOUNT => 1000));
```

```

-- =====
-- DEMONSTRATION 1: Result Cache (QUERY RESULT REUSE node)
-- =====

-- Disable result cache for first run to see actual performance
ALTER SESSION SET USE_CACHED_RESULT = FALSE;

-- First run: actual table scan (see TableScan node in Query Profile)
SELECT region, SUM(amount) AS total_sales, COUNT(*) AS num_orders
FROM sales_fact
GROUP BY region
ORDER BY total_sales DESC;
-- Open Query Profile → look at: TableScan node, Partitions Scanned, Bytes Scanned

-- Re-enable result cache and run the IDENTICAL query
ALTER SESSION SET USE_CACHED_RESULT = TRUE;

SELECT region, SUM(amount) AS total_sales, COUNT(*) AS num_orders
FROM sales_fact
GROUP BY region
ORDER BY total_sales DESC;
-- Query Profile: single node "QUERY RESULT REUSE" – millisecond response!

-- Non-deterministic function BREAKS result cache
SELECT region, SUM(amount) AS total_sales, CURRENT_TIMESTAMP() AS at
FROM sales_fact
GROUP BY region
ORDER BY total_sales DESC;
-- Query Profile: FULL scan again – CURRENT_TIMESTAMP() prevents cache use

-- =====
-- DEMONSTRATION 2: Metadata Cache (METADATA-BASED RESULT node)
-- =====

-- These queries resolve from metadata (no warehouse compute):
ALTER SESSION SET USE_CACHED_RESULT = FALSE; -- Disable result cache to show metadata
SELECT COUNT(*) FROM sales_fact;
-- Query Profile: "METADATA-BASED RESULT" – milliseconds, 0 bytes scanned

SELECT MIN(sale_date), MAX(sale_date) FROM sales_fact;
-- Query Profile: "METADATA-BASED RESULT"

-- Adding a WHERE clause forces actual data scan (metadata can't answer filtered count)
SELECT COUNT(*) FROM sales_fact WHERE amount > 4000;
-- Query Profile: actual TableScan node (must scan to evaluate the filter)
-- Compare time and bytes scanned!

ALTER SESSION SET USE_CACHED_RESULT = TRUE;

-- =====
-- DEMONSTRATION 3: Pruning – before and after clustering key
-- =====

ALTER SESSION SET USE_CACHED_RESULT = FALSE;

-- Query with date filter (no clustering key yet)
SELECT sale_id, region, amount
FROM sales_fact

```

```

WHERE sale_date = CURRENT_DATE() - 90;
-- Query Profile → TableScan → Partitions Scanned ≈ Partitions Total (poor pruning!)
-- Note the values before we add clustering

-- Add a clustering key on sale_date
ALTER TABLE sales_fact CLUSTER BY (sale_date);

-- Clustering takes time in the background. For demo, query immediately:
SELECT sale_id, region, amount
FROM sales_fact
WHERE sale_date = CURRENT_DATE() - 90;
-- After clustering settles: Partitions Scanned << Partitions Total (effective pruning)
-- Check: SYSTEM$CLUSTERING_INFORMATION
SELECT SYSTEM$CLUSTERING_INFORMATION('PERF_DEMO.QUERY_PROFILE_LAB.SALES_FACT', '(sale_id, region, amount)')
FROM sales_fact;

-- =====
-- DEMONSTRATION 4: Data Spilling (scale down to force spill)
-- =====

-- Resize warehouse smaller to increase spill likelihood on large sort
-- ALTER WAREHOUSE learn_wh SET WAREHOUSE_SIZE = 'X-SMALL';

-- Run a memory-intensive query (large sort + window function)
SELECT
    sale_id,
    sale_date,
    region,
    amount,
    SUM(amount) OVER (PARTITION BY region ORDER BY sale_date
                      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM sales_fact
ORDER BY region, sale_date, sale_id;
-- Query Profile: check "Bytes Spilled to Local Storage" in stats
-- If X-SMALL, may see spill to local storage

-- Resize back to normal
-- ALTER WAREHOUSE learn_wh SET WAREHOUSE_SIZE = 'X-SMALL'; -- restore

-- =====
-- DEMONSTRATION 5: Join Explosion (Cartesian Product)
-- =====

-- SAFE DEMO – join without proper condition (intentional error for learning)
-- WARNING: This will produce a large result set!
-- The LIMIT keeps it manageable

-- Normal join (correct):
SELECT f.sale_id, f.amount, p.product_name, p.category
FROM sales_fact f
JOIN product_dim p ON f.product_id = p.product_id
LIMIT 100;
-- Query Profile: Join node output rows ≈ input rows (no explosion)

-- Dangerous join (missing join condition = cross join):
-- SELECT f.sale_id, p.product_name
-- FROM sales_fact f
-- CROSS JOIN product_dim p
-- LIMIT 100;

```

```

-- Query Profile: Join node output = 500,000 × 1,000 = 500 MILLION rows!
-- Don't run without LIMIT – it will be extremely slow

-- =====
-- DEMONSTRATION 6: Query History for performance investigation
-- =====

USE ROLE ACCOUNTADMIN;

-- Recent queries with performance metrics
SELECT
    QUERY_ID,
    USER_NAME,
    WAREHOUSE_NAME,
    LEFT(QUERY_TEXT, 80)          AS query_preview,
    TOTAL_ELAPSED_TIME/1000       AS elapsed_sec,
    BYTES_SCANNED/1048576         AS mb_scanned,
    PARTITIONS_SCANNED,
    PARTITIONS_TOTAL,
    ROUND(PARTITIONS_SCANNED * 100.0 / NULLIF(PARTITIONS_TOTAL,0), 1) AS pct_partitions_scanned,
    BYTES_SPILLED_TO_LOCAL_STORAGE/1048576 AS mb_spilled_local,
    BYTES_SPILLED_TO_REMOTE_STORAGE/1048576 AS mb_spilled_remote,
    COMPILATION_TIME/1000        AS compile_sec,
    EXECUTION_TIME/1000         AS execute_sec
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE START_TIME >= DATEADD('hour', -2, CURRENT_TIMESTAMP())
    AND EXECUTION_STATUS = 'SUCCESS'
    AND DATABASE_NAME = 'PERF_DEMO'
ORDER BY TOTAL_ELAPSED_TIME DESC
LIMIT 20;

-- Find queries with highest spill to REMOTE storage (worst offenders)
SELECT QUERY_ID, WAREHOUSE_NAME, LEFT(QUERY_TEXT,80),
    BYTES_SPILLED_TO_REMOTE_STORAGE/1073741824 AS gb_spilled_remote
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE START_TIME >= DATEADD('day', -7, CURRENT_TIMESTAMP())
    AND BYTES_SPILLED_TO_REMOTE_STORAGE > 0
ORDER BY BYTES_SPILLED_TO_REMOTE_STORAGE DESC
LIMIT 10;

-- Find queries with worst pruning (scanned > 80% of all partitions)
SELECT QUERY_ID, LEFT(QUERY_TEXT,80),
    PARTITIONS_SCANNED, PARTITIONS_TOTAL,
    ROUND(PARTITIONS_SCANNED * 100.0 / NULLIF(PARTITIONS_TOTAL,0), 1) AS pct_partitions_scanned
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE START_TIME >= DATEADD('day', -7, CURRENT_TIMESTAMP())
    AND PARTITIONS_TOTAL > 50
    AND PARTITIONS_SCANNED * 1.0 / NULLIF(PARTITIONS_TOTAL, 0) > 0.8
ORDER BY PARTITIONS_SCANNED DESC
LIMIT 20;

```


☒ Topic 3.1 — 20 Practice Questions: Query Profile

Q1. In the Snowflake Query Profile, a TableScan node shows `Partitions Scanned = 15` and `Partitions Total = 12,400`. What does this tell you?

- A) The query is inefficient — it scanned almost all partitions
- B) Partition pruning is highly effective — only 15 of 12,400 partitions needed to be read
- C) The table only has 15 micro-partitions and the rest are empty
- D) The query used the Metadata Cache to skip 12,385 partitions

Q2. A Query Profile shows a single node labeled "QUERY RESULT REUSE" and the query completes in 50 milliseconds. What happened?

- A) The warehouse's local SSD cache served the result
- B) The Metadata Cache answered the query from partition statistics
- C) The Result Cache returned the pre-computed answer from a previous identical query
- D) Snowflake automatically optimized the query to read fewer partitions

Q3. "Bytes Spilled to Remote Storage" in a query's statistics indicates which problem?

- A) The query was sent to a remote Snowflake account for processing
- B) Intermediate results exceeded the warehouse's available memory AND local SSD, overflowing to cloud storage — severely degrading performance
- C) Snowflake migrated data to a remote region for compliance reasons
- D) The query accessed an external table in remote cloud storage

Q4. In a Join node of the Query Profile, the input tables have 500,000 and 200 rows respectively, but the join output is 4 billion rows. What is this problem called?

- A) Data Spilling
- B) Inefficient Pruning
- C) Exploding Join (Join Explosion)
- D) Query Queuing

Q5. Which of the following Query Profile node types indicates a query was answered WITHOUT using any virtual warehouse compute? (*Select ALL that apply*)

- A) TableScan
- B) QUERY RESULT REUSE
- C) METADATA-BASED RESULT
- D) Filter
- E) Aggregate

Q6. A user runs `SELECT COUNT(*) FROM orders` and sees the result in 30 milliseconds with 0 bytes scanned in the Query Profile. Which cache was used?

- A) Result Cache — the query was previously run
- B) Metadata Cache — row count is stored in Cloud Services micro-partition metadata
- C) Warehouse Local Cache — the table is small enough to fit in SSD
- D) Query Compilation Cache — SQL parsing was reused

Q7. Adding `CURRENT_TIMESTAMP()` to a `SELECT` clause prevents which type of caching?

- A) Metadata Cache — `CURRENT_TIMESTAMP` changes partition statistics
- B) Warehouse Local Cache — time-dependent queries can't be cached on SSD
- C) Result Cache — non-deterministic functions like `CURRENT_TIMESTAMP()` produce different results each call, so the cached result can't be reused
- D) Query Compilation Cache — prevents SQL parsing from being reused

Q8. In the Query Profile's Exchange node, what does a **BROADCAST** exchange operation indicate?

- A) A large table is being distributed across all warehouse nodes for parallel processing
- B) A small table is being sent (broadcast) to all warehouse nodes so each node has a local copy for the join
- C) Data is being sent to another Snowflake account via data sharing
- D) The query result is being broadcast to all users who ran the same query

Q9. "High Transaction Blocked Time" at the top of the Query Profile timeline indicates which problem?

- A) The query was blocked by a lock conflict on the table
- B) The database connection timed out and had to reconnect
- C) The query spent significant time waiting in the warehouse queue due to concurrency limits
- D) The query's compilation step was slower than expected

Q10. What does `Bytes Spilled to Local Storage` mean in a Snowflake Query Profile?

- A) Data was written to the table's Fail-Safe storage
- B) Intermediate query results exceeded available in-memory buffer and overflowed to the warehouse's local SSD — less severe than remote spill
- C) The query output was saved to a local internal stage
- D) Files were loaded from an internal stage during a `COPY INTO` operation

Q11. A developer runs the same `SELECT` query twice. The first run takes 45 seconds; the second run takes 0.2 seconds with the same result. `USE_CACHED_RESULT = TRUE`. What most

likely explains the 0.2-second response on the second run?

- A) Snowflake optimized the query plan on the second attempt
- B) The warehouse was scaled up automatically between the two runs
- C) The Result Cache served the second query — the result was cached from the first run
- D) The Warehouse Local Cache loaded all micro-partitions into SSD during the first run and the second run read from SSD

Q12. Which of the following would BREAK the Result Cache hit (i.e., force a full re-scan)?
(Select ALL that apply)

- A) Running the identical query text with the same parameters
- B) A DML operation (INSERT/UPDATE/DELETE) occurs on the queried table between the two runs
- C) Adding a column to the SELECT that contains RANDOM ()
- D) Running the query from a different virtual warehouse
- E) The 24-hour cache expiration period elapses without the query being re-run

Q13. In the Query Profile, what does a high "Percentage of Query Time" on an Aggregate node typically suggest?

- A) The GROUP BY columns are not clustered, forcing a full scan
- B) The aggregation is working on too many rows — potentially due to a missing or late-applied filter
- C) The warehouse is too large, causing unnecessary parallelism overhead
- D) The aggregation is using the Result Cache inefficiently

Q14. How long is query history available in Snowsight's Query History UI for accessing the Query Profile?

- A) 24 hours
- B) 7 days
- C) 14 days
- D) 30 days

Q15. Which ACCOUNT_USAGE view contains the BYTES_SPILLED_TO_REMOTE_STORAGE and PARTITIONS_SCANNED columns used for programmatic performance analysis?

- A) SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY
- B) SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
- C) SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
- D) SNOWFLAKE.ACCOUNT_USAGE.STORAGE_USAGE

Q16. A query runs `SELECT MIN(amount), MAX(amount) FROM sales_fact WHERE region = 'North'`. Will this be answered by the Metadata Cache?

- A) Yes — MIN and MAX are always answered from the Metadata Cache
- B) No — the WHERE clause requires scanning data to filter by `region='North'` before computing MIN/MAX; it cannot be served from metadata alone
- C) Yes — the WHERE clause is compiled away during optimization
- D) It depends on whether the clustering key is on the `region` column

Q17. What is the FIRST step a developer should take when debugging a slow Snowflake query?

- A) Immediately scale up the warehouse to the largest size to fix the problem
- B) Open the Query Profile for the slow query and identify which node is consuming the most time (Most Expensive Nodes)
- C) Disable all caches so the query runs fresh data
- D) Drop and recreate the table with a new clustering key

Q18. Snowflake's Query Insights panel in Snowsight shows `INEFFICIENT_JOIN_CONDITION`. What does this mean?

- A) The join was performed between tables from different databases
- B) The join condition contains a complex expression that is evaluated AFTER the tables are joined — rather than being pushed down as a pre-filter
- C) The join used a non-equality operator (e.g., `>=`) which is not supported
- D) The join result exceeded the warehouse's memory limit

Q19. A query scanned 5,000 partitions out of 5,000 total (100% scan rate) but ran in 3 seconds with no spilling. What is the MOST APPROPRIATE conclusion?

- A) The query has a critical performance problem that must be fixed immediately
- B) Full partition scans are always a problem regardless of query time
- C) 100% partition scan may be acceptable if the query is an infrequent full-table scan or the data volume is small; not all full scans need a clustering key fix
- D) The Result Cache must have been used since the query ran fast

Q20. Which Query Profile node type appears when using a WITH clause (CTE) that is referenced multiple times in a query?

- A) ExternalScan — CTEs read from temporary external storage
- B) WithReference / WithClause — Snowflake materializes the CTE once and references it for each use
- C) Exchange — CTEs exchange data between warehouse nodes
- D) Aggregate — CTEs are pre-aggregated before being referenced

SUBDOMAIN 3.2 — Virtual Warehouse Configurations

☒ Layman's Explanation

A Virtual Warehouse is your compute engine — the workers who actually execute your SQL queries. Snowflake separates compute from storage, so you can have as many or as few "work teams" (warehouses) running as you need.

- **Warehouse Size** = How many people are on the team. X-Small = 1 worker. 6X-Large = 512 workers. More workers = faster queries on complex/large jobs. But also costs more credits.
- **Scale Up** = Hiring more people for the SAME team to handle a single complex task faster (e.g., a monster 10TB query)
- **Scale Out** = Opening MORE IDENTICAL teams to handle more simultaneous customers (e.g., 500 users all running dashboards at once)
- **Auto-Suspend** = Send workers home when they're idle to avoid paying them for doing nothing
- **Auto-Resume** = Workers automatically show up when a new query arrives — zero manual effort

☒ Technical Explanation

Warehouse Sizes and Credits^[8][2]

Each warehouse size doubles the compute (and cost) of the previous size:

Size	Credits/Hour	Relative Power	Nodes per Cluster
X-Small (XS)	1	1×	1
Small (S)	2	2×	2
Medium (M)	4	4×	4
Large (L)	8	8×	8
X-Large (XL)	16	16×	16
2X-Large	32	32×	32
3X-Large	64	64×	64
4X-Large	128	128×	128
5X-Large	256	256×	256
6X-Large	512	512×	512

Billing rules (critical for exam):

- Billed per second with a 60-second minimum per start
- "Start" = warehouse transitions from SUSPENDED to RUNNING
- Resizing a RUNNING warehouse: no credit interruption — charged at new size rate going forward
- Auto-suspend: default = 10 minutes; can be set from 1 minute to unlimited
- Cloud Services credits are charged separately — if Cloud Services credits exceed 10% of daily compute credits, the excess is billed

Warehouse Types^[9][8]

Type	Description	Use Case
Standard	Default warehouse type. Balances cost and performance.	General-purpose queries, ETL, BI
Snowpark-Optimized	16× more memory per node than Standard. Higher credit cost.	Python/Java/Scala Snowpark ML training jobs, large in-memory DataFrame operations

Edge case: Snowpark-Optimized warehouses are single-cluster ONLY (no multi-cluster scaling). They are designed for memory-intensive single jobs, not concurrency.

Multi-Cluster Warehouses^[10][11][^9]

A Multi-Cluster Warehouse runs multiple identical warehouse clusters in parallel. Each cluster is a full copy of the warehouse. This solves the **concurrency** problem — too many simultaneous users or queries.

Key parameters:

- MIN_CLUSTER_COUNT — minimum number of clusters always running (sets minimum cost floor)
- MAX_CLUSTER_COUNT — maximum number of clusters Snowflake can auto-start

Multi-Cluster Modes:

Mode	How It Works	When to Use
Auto-Scale (default)	MIN_CLUSTER_COUNT ≠ MAX_CLUSTER_COUNT. Snowflake automatically starts/stops clusters based on load.	Most BI and dashboard workloads — variable concurrency
Maximized	MIN_CLUSTER_COUNT = MAX_CLUSTER_COUNT. ALL clusters always running.	Consistently high, predictable concurrent workloads (always need maximum capacity)

Scaling Policies in Auto-Scale Mode:[^11]

Policy	New Cluster Starts When...	Cluster Shuts Down When...	Best For
Standard (default)	A query is queued OR Snowflake estimates resources insufficient	After a sustained period of low load (clusters shut down one at a time)	Most workloads — prevents queuing
Economy	Conserves credits — starts new cluster only when queuing is clearly established	Shuts down faster when load drops	Cost-sensitive environments; acceptable slight queuing

Critical exam traps about multi-cluster warehouses:[¹¹][9]

- Multi-cluster solves CONCURRENCY problems — NOT the "my single query is slow" problem
- For a slow single query → Scale UP the warehouse size
- For many concurrent users queuing → Scale OUT (multi-cluster)
- Requires Enterprise edition or higher
- Auto-suspend applies to the ENTIRE multi-cluster warehouse — not individual clusters
- Individual clusters are started/stopped by Snowflake automatically in Auto-Scale mode
- Credit cost = sum of all running clusters × warehouse size credits/hour

Warehouse Settings Reference[¹²][8][^9]

Parameter	Default	Description
AUTO_SUSPEND	600 seconds (10 min)	Seconds of inactivity before auto-suspend
AUTO_RESUME	TRUE	Resume automatically when a query is submitted
MAX_CONCURRENCY_LEVEL	8	Max concurrent SQL statements per cluster. When reached: single-cluster queues; multi-cluster Auto-scale starts new cluster
STATEMENT_QUEUED_TIMEOUT_IN_SECONDS	0 (unlimited)	Max time a query waits in queue before it's cancelled
STATEMENT_TIMEOUT_IN_SECONDS	0 (unlimited)	Max runtime for any single statement
SCALING_POLICY	STANDARD	Auto-Scale behavior: STANDARD or ECONOMY

☒ Hands-On Queries: Virtual Warehouse Configuration

```
-- =====
-- PART 1: Create and Configure Warehouses
-- =====
USE ROLE SYSADMIN;
```

```

-- Standard Single-Cluster Warehouse
CREATE OR REPLACE WAREHOUSE etl_warehouse
    WAREHOUSE_TYPE      = STANDARD
    WAREHOUSE_SIZE      = 'MEDIUM'
    AUTO_SUSPEND        = 300          -- Suspend after 5 minutes of inactivity
    AUTO_RESUME         = TRUE
    COMMENT             = 'ETL processing warehouse';

-- Multi-Cluster Warehouse in Auto-Scale Mode
CREATE OR REPLACE WAREHOUSE bi_warehouse
    WAREHOUSE_TYPE      = STANDARD
    WAREHOUSE_SIZE      = 'SMALL'
    MIN_CLUSTER_COUNT   = 1           -- Always have at least 1 cluster
    MAX_CLUSTER_COUNT   = 5           -- Can scale up to 5 clusters
    SCALING_POLICY      = 'STANDARD'  -- Prevent queuing
    AUTO_SUSPEND        = 120         -- 2 minutes (BI users want fast response)
    AUTO_RESUME         = TRUE
    COMMENT             = 'BI dashboards and ad-hoc analytics';

-- Maximized Multi-Cluster (ALL clusters always running)
CREATE OR REPLACE WAREHOUSE high_volume_warehouse
    WAREHOUSE_TYPE      = STANDARD
    WAREHOUSE_SIZE      = 'LARGE'
    MIN_CLUSTER_COUNT   = 3           -- Always 3 clusters running
    MAX_CLUSTER_COUNT   = 3           -- Never more than 3 (Maximized = MIN = MAX)
    AUTO_SUSPEND        = 600
    AUTO_RESUME         = TRUE
    COMMENT             = 'High-volume report generation - constant concurrency';

-- Snowpark-Optimized Warehouse (for ML workloads)
CREATE OR REPLACE WAREHOUSE ml_training_warehouse
    WAREHOUSE_TYPE      = 'SNOWPARK-OPTIMIZED'
    WAREHOUSE_SIZE      = 'MEDIUM'   -- 16x more memory than standard MEDIUM
    AUTO_SUSPEND        = 60          -- Suspend quickly when ML job finishes
    AUTO_RESUME         = TRUE
    COMMENT             = 'Snowpark ML model training - high memory workloads';

-- =====
-- PART 2: View and Modify Warehouse Settings
-- =====

-- List all warehouses
SHOW WAREHOUSES;

-- Get details of a specific warehouse
DESC WAREHOUSE bi_warehouse;

-- Scale UP a warehouse (e.g., ETL job needs more power)
ALTER WAREHOUSE etl_warehouse SET WAREHOUSE_SIZE = 'LARGE';
-- Takes effect immediately for new queries; in-flight queries continue on old size

-- Change multi-cluster settings
ALTER WAREHOUSE bi_warehouse SET
    MIN_CLUSTER_COUNT = 2           -- Keep at least 2 clusters running
    MAX_CLUSTER_COUNT = 8           -- Allow up to 8 during peaks
    SCALING_POLICY    = 'ECONOMY';  -- Save credits when possible

```



```

-- Adjust timeout settings
ALTER WAREHOUSE bi_warehouse SET
    STATEMENT_QUEUED_TIMEOUT_IN_SECONDS = 30 -- Cancel if queued >30 sec
    STATEMENT_TIMEOUT_IN_SECONDS        = 300; -- Cancel if running >5 min

-- =====
-- PART 3: Suspend and Resume
-- =====

-- Manual suspend (immediately stops billing if no queries running)
ALTER WAREHOUSE etl_warehouse SUSPEND;

-- Manual resume (use WAREHOUSE activates it automatically)
ALTER WAREHOUSE etl_warehouse RESUME;

-- Use a warehouse (auto-resumes if suspended due to AUTO_RESUME = TRUE)
USE WAREHOUSE etl_warehouse;

-- Check warehouse status
SELECT
    name,
    state,                -- STARTED, SUSPENDED, RESIZING
    size,
    min_cluster_count,
    max_cluster_count,
    scaling_policy,
    auto_suspend,
    auto_resume,
    running,              -- Current number of running clusters
    queued,               -- Queries currently queued
    resource_monitor
FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()));
-- (After SHOW WAREHOUSES above)

SHOW WAREHOUSES;
-- Look at: STATE column (STARTED/SUSPENDED), RUNNING, QUEUED columns

-- =====
-- PART 4: Warehouse Load Monitoring
-- =====

USE ROLE ACCOUNTADMIN;

-- Monitor warehouse load over last 24 hours
SELECT
    WAREHOUSE_NAME,
    DATE_TRUNC('hour', START_TIME) AS hour,
    COUNT(*) AS total_queries,
    AVG(TOTAL_ELAPSED_TIME/1000) AS avg_sec,
    MAX(TOTAL_ELAPSED_TIME/1000) AS max_sec,
    SUM(CASE WHEN TRANSACTION_BLOCKED_TIME > 0 THEN 1 ELSE 0 END) AS queued_queries,
    AVG(TRANSACTION_BLOCKED_TIME/1000) AS avg_queue_sec
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE START_TIME >= DATEADD('day', -1, CURRENT_TIMESTAMP())
    AND WAREHOUSE_NAME IN ('BI_WAREHOUSE', 'ETL_WAREHOUSE')
GROUP BY 1, 2
ORDER BY 1, 2;

```

```

-- Credits consumed by warehouse per day (last 14 days)
SELECT
    WAREHOUSE_NAME,
    DATE(START_TIME) AS usage_date,
    ROUND(SUM(CREDITS_USED), 3) AS total_credits,
    COUNT(DISTINCT DATE_TRUNC('hour', START_TIME)) AS active_hours
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE START_TIME >= DATEADD('day', -14, CURRENT_TIMESTAMP())
GROUP BY 1, 2
ORDER BY 2 DESC, 3 DESC;

-- =====
-- PART 5: Warehouse Performance Settings
-- =====

-- Check all current warehouse parameters
SHOW PARAMETERS IN WAREHOUSE bi_warehouse;

-- Set concurrency level (max parallel statements per cluster)
ALTER WAREHOUSE bi_warehouse SET MAX_CONCURRENCY_LEVEL = 12;
-- Default is 8. When this is hit: single-cluster queues; multi-cluster spins up new

-- Clean up
DROP WAREHOUSE IF EXISTS etl_warehouse;
DROP WAREHOUSE IF EXISTS high_volume_warehouse;
DROP WAREHOUSE IF EXISTS ml_training_warehouse;
-- Keep bi_warehouse for next sections

```

☒ Topic 3.2 — 20 Practice Questions: Warehouse Configurations

Q1. A Medium warehouse runs for 35 seconds and then is suspended. How many credits are billed?

- A) $35/3600 \times 4$ credits (proportional to seconds used)
- B) $60/3600 \times 4$ credits — the 60-second minimum billing rule applies
- C) 4 credits — one full hour is always billed regardless
- D) 0 credits — less than 1 minute of usage is not billed

Q2. What is the key difference between "Scale Up" and "Scale Out" in Snowflake?

- A) Scale Up means adding more databases; Scale Out means adding more schemas
- B) Scale Up increases warehouse SIZE (more compute per cluster) to make a single query faster; Scale Out adds more CLUSTERS (multi-cluster) to handle more concurrent users
- C) Scale Up applies to Standard warehouses; Scale Out applies to Enterprise warehouses only
- D) Scale Up and Scale Out are synonymous in Snowflake — both add more compute

Q3. A company has 300 concurrent BI users all queuing on the same warehouse during business hours. What is the correct solution?

- A) Scale the warehouse up to 6X-Large to reduce individual query time
- B) Enable multi-cluster warehouse (Scale Out) so Snowflake can automatically start more clusters to serve concurrent users
- C) Create 300 separate warehouses — one per user
- D) Increase `STATEMENT_QUEUED_TIMEOUT_IN_SECONDS` to allow longer queue times

Q4. A multi-cluster warehouse has `MIN_CLUSTER_COUNT = 2` and `MAX_CLUSTER_COUNT = 2`. What mode is this?

- A) Auto-Scale mode — dynamically adjusts between 2 and 2 clusters
- B) Maximized mode — both minimum and maximum are equal, so all defined clusters always run
- C) Economy mode — 2 clusters is the minimum for Economy scaling
- D) Single-Cluster mode — 2 clusters is treated as single-cluster when `MIN = MAX`

Q5. What happens to a warehouse's local SSD cache when the warehouse is RESIZED?

- A) The cache is preserved and migrated to the new warehouse nodes
- B) The cache is compressed to fit the new node configuration
- C) The cache is completely CLEARED — new nodes have empty caches, so the first queries after resize are slower
- D) The cache is backed up to cloud storage and restored after resize

Q6. A Snowpark-Optimized warehouse has how many times more memory per node compared to a Standard warehouse of the same size?

- A) 2× more memory
- B) 4× more memory
- C) 8× more memory
- D) 16× more memory

Q7. In Auto-Scale mode with STANDARD scaling policy, when does Snowflake start an additional cluster?

- A) Only after 5 consecutive queries have been queued
- B) When a query is queued OR when Snowflake estimates current clusters don't have enough resources for additional queries
- C) Only at the beginning of every hour during business hours
- D) Only when manually triggered by ACCOUNTADMIN

Q8. Which warehouse type is ONLY supported as a single-cluster warehouse (no multi-cluster scaling)?

- A) Standard

- B) Economy
- C) Snowpark-Optimized
- D) Enterprise

Q9. What does `AUTO_RESUME = TRUE` do?

- A) Restarts a crashed warehouse automatically after a system failure
- B) Automatically resumes a suspended warehouse when a new SQL query is submitted to it
- C) Automatically scales the warehouse up when query performance drops below a threshold
- D) Resumes all suspended warehouses at the same time every morning

Q10. A data scientist needs to train a large ML model using Snowpark Python. The training job requires processing a 500GB DataFrame entirely in memory. Which warehouse type is most appropriate?

- A) Standard X-Large — biggest Standard warehouse
- B) Multi-Cluster Standard Medium — scale out for concurrency
- C) Snowpark-Optimized Medium — provides 16× more memory for in-memory processing
- D) Standard 2X-Large — doubles the standard node count

Q11. In Auto-Scale mode, the `ECONOMY` scaling policy differs from `STANDARD` in what way?

- A) Economy uses smaller warehouse sizes when starting new clusters
- B) Economy waits longer before starting new clusters and shuts them down faster — conserves credits at the expense of slightly more queuing
- C) Economy only supports a maximum of 3 clusters regardless of `MAX_CLUSTER_COUNT`
- D) Economy is the default scaling policy for all new warehouses

Q12. A warehouse has `AUTO_SUSPEND = 60`. A query finishes at 3:00 PM. When does the warehouse auto-suspend?

- A) Immediately after the last query finishes
- B) At 3:01 PM — 60 seconds after the last query completed
- C) At 4:00 PM — 60 minutes after the last activity
- D) It never suspends — `AUTO_SUSPEND = 60` is the minimum setting and doesn't actually enable suspension

Q13. What is `MAX_CONCURRENCY_LEVEL` on a warehouse?

- A) The maximum number of clusters in a multi-cluster warehouse
- B) The maximum number of SQL statements that can execute concurrently on a single cluster; when reached, queries queue (single-cluster) or a new cluster starts (multi-

cluster auto-scale)

- C) The maximum number of users that can connect to the warehouse simultaneously
- D) The maximum number of virtual warehouses allowed in one Snowflake account

Q14. An ETL warehouse is running an overnight batch job. The job is taking 4 hours. The developer thinks scaling the warehouse UP from LARGE to 2X-LARGE would help. What effect does resizing a RUNNING warehouse have on the current query?

- A) The current query is immediately cancelled and must be re-run on the new size
- B) The resize takes effect immediately — the current query gains the benefit of the new size
- C) The resize queues and only takes effect after the current query completes; new queries start on the new size
- D) The warehouse must be suspended first — you cannot resize a running warehouse

Q15. How are credits billed for a multi-cluster warehouse with `MIN_CLUSTER_COUNT = 2`, `MAX_CLUSTER_COUNT = 4`, and `WAREHOUSE_SIZE = MEDIUM` when 3 clusters are actively running?

- A) Billed for 1 cluster (base rate only)
- B) Billed for 2 clusters (minimum count always)
- C) Billed for 3 clusters × Medium rate (4 credits each) = 12 credits/hour
- D) Billed for 4 clusters (maximum) regardless of active clusters

Q16. What happens when `STATEMENT_QUEUED_TIMEOUT_IN_SECONDS = 30` and a query has been waiting in the queue for 31 seconds?

- A) The warehouse automatically adds a cluster to process the query
- B) The query is automatically cancelled and the user receives a timeout error
- C) The query priority is elevated to the front of the queue
- D) The warehouse is automatically resized to accommodate the query

Q17. A company wants ALL warehouse clusters in a multi-cluster warehouse to be running at ALL times, regardless of query load. Which mode achieves this?

- A) Auto-Scale with STANDARD policy
- B) Auto-Scale with ECONOMY policy
- C) Maximized mode — set `MIN_CLUSTER_COUNT = MAX_CLUSTER_COUNT`
- D) Single-Cluster mode with `MAX_CONCURRENCY_LEVEL = MAX`

Q18. Cloud Services credits are FREE as long as they don't exceed what threshold?

- A) 5% of daily compute (warehouse) credits
- B) 10% of daily compute (warehouse) credits
- C) 15% of daily compute (warehouse) credits

- D) 25% of daily compute (warehouse) credits

Q19. Which of the following are TRUE about multi-cluster warehouses? (*Select ALL that apply*)

- A) Multi-cluster warehouses solve concurrency problems — too many simultaneous queries
- B) Multi-cluster warehouses solve single-query performance problems — one complex query runs faster
- C) Auto-suspend in multi-cluster applies to the entire warehouse, not individual clusters
- D) Multi-cluster warehouses require Enterprise edition or higher
- E) In Maximized mode, all clusters defined by MAX_CLUSTER_COUNT are always running

Q20. A warehouse has `AUTO_SUSPEND = 300` and `AUTO_RESUME = TRUE`. A user submits a query when the warehouse is suspended. What happens?

- A) The query fails with an error: "Warehouse is suspended, please resume manually"
- B) The warehouse automatically resumes, the query waits briefly for the warehouse to start, then executes normally
- C) The query is cached and runs when the warehouse is manually resumed
- D) The query runs on a different active warehouse in the account

SUBDOMAIN 3.3 — Virtual Warehouse Performance Tools

☒ Layman's Explanation

After you've built your warehouse, Snowflake gives you tools to ask: "Is this warehouse working well? Is it being used efficiently? Should I change something?"

- **Monitoring Warehouse Loads** = Checking the dashboard to see when the warehouse is busiest, how long queries wait in queue, and what the idle time looks like
- **Scaling Up vs. Scaling Out** = The classic choice: bigger single engine vs. more parallel engines. Right answer depends on the problem.
- **Query Acceleration Service (QAS)** = A turbo booster for occasional super-heavy queries. Instead of sizing your entire warehouse for the worst-case query, QAS handles the heavy parts using borrowed serverless compute.

☒ Technical Explanation

Monitoring Warehouse Loads^{[8][3]}

In Snowsight:

- Admin → Warehouses → click a warehouse → shows usage timeline, queuing, and credits consumed
- Activity → Query History → filter by warehouse, see per-query performance

In ACCOUNT_USAGE:

- WAREHOUSE_METERING_HISTORY — hourly credit consumption per warehouse
- WAREHOUSE_LOAD_HISTORY (available via Snowsight) — query count, queue depth over time
- QUERY_HISTORY — per-query metrics including TRANSACTION_BLOCKED_TIME (queue time)

Key signals to monitor:

Signal	Interpretation	Action
High TRANSACTION_BLOCKED_TIME	Queries queuing — warehouse undersized for concurrency	Add clusters (multi-cluster) or add dedicated warehouse
High BYTES_SPILLED_TO_REMOTE_STORAGE	Queries running out of memory	Scale UP warehouse
Low credit usage but high query count	Queries are fast and efficient — warehouse may be oversized	Scale down
Many queries running 60-120 seconds	Hitting the 60-second minimum billing repeatedly on small queries	Consider Serverless tasks or a smaller warehouse

Scaling Up vs. Scaling Out — Decision Framework^{[2][9][^8]}

Scenario	Solution	Why
Single large query is slow (e.g., complex JOIN on 10TB table)	Scale UP (increase warehouse size)	More memory per node + more parallelism per query
200 users all running dashboards simultaneously; queries are fast but queuing	Scale OUT (multi-cluster)	Each cluster serves a set of users; no shared queue
ETL pipeline needs to load and transform 1TB faster	Scale UP	More compute nodes = faster parallel loading
Morning reporting rush — 50 users all run the same report at 8 AM	Scale OUT	Concurrency burst, not query complexity
Query spills to remote storage	Scale UP	More memory = less spilling
Background task runs at 3 AM with no users — reduce overnight cost	Auto-suspend or smaller warehouse	Zero concurrency need

Query Acceleration Service (QAS) — Deep Dive^{[13][14]}

What it is: A service that offloads portions of eligible queries from the assigned virtual warehouse to a pool of shared serverless compute resources — Snowflake's infrastructure, not yours.

How it works:

1. You enable QAS on a warehouse: `ALTER WAREHOUSE my_wh SET ENABLE_QUERY_ACCELERATION = TRUE`
2. Snowflake evaluates each query submitted to that warehouse for "eligibility" — will QAS actually help?
3. For eligible queries, portions of the query (large scans, certain operations) are sent to the shared serverless pool
4. Results are combined and returned to the user via the original warehouse
5. If QAS doesn't help a query, it runs entirely on the assigned warehouse as normal

`QUERY_ACCELERATION_MAX_SCALE_FACTOR`:^[14]

- Controls how many shared serverless resources QAS can use relative to the warehouse size
- Range: 0 to 100 (default: 8)
- Scale factor 8 = QAS can use up to 8× the warehouse's compute capacity in shared resources
- Setting it to 0 disables QAS eligibility without removing the setting

Which queries benefit from QAS:^{[13][14]}

- Ad-hoc analytical queries with unpredictable data volume
- Queries with large scans AND highly selective filters (filters that eliminate most rows)
- Star schema queries on large fact tables
- Queries that are "outliers" — most queries on that warehouse are fast but this one is slow

Which queries do NOT benefit from QAS:^[14]

- Small, fast queries (QAS overhead isn't worth it)
- Queries that are already fast on the warehouse
- Data loading / DML operations
- Queries that only read from a tiny number of micro-partitions

Checking QAS eligibility:

```
-- Check if queries are being accelerated
SELECT QUERY_ID, QUERY_TYPE, QAS_CREDITS_USED
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
```



```
WHERE QAS_CREDITS_USED > 0
ORDER BY QAS_CREDITS_USED DESC;
```

Cost: QAS uses **serverless compute credits** (separate from warehouse credits). You pay per second of serverless compute used during acceleration.

SOS + QAS Complementarity (exam favorite):^{[15][13]}

- Search Optimization Service (SOS) runs **FIRST** → prunes micro-partitions
- QAS runs **SECOND** → accelerates scanning of the remaining (non-pruned) micro-partitions
- They are fully compatible — both can work on the same query simultaneously

☒ Hands-On Queries: Warehouse Performance Tools

```
-- =====
-- PART 1: Monitor Warehouse Load
-- =====

USE ROLE ACCOUNTADMIN;

-- Warehouse activity summary last 7 days
SELECT
    WAREHOUSE_NAME,
    DATE(START_TIME) AS usage_date,
    COUNT(*) AS total_queries,
    SUM(CASE WHEN TRANSACTION_BLOCKED_TIME > 5000 THEN 1 ELSE 0 END) AS queued_queries,
    ROUND(AVG(TOTAL_ELAPSED_TIME)/1000, 2) AS avg_elapsed_sec,
    ROUND(MAX(TOTAL_ELAPSED_TIME)/1000, 2) AS max_elapsed_sec,
    ROUND(AVG(TRANSACTION_BLOCKED_TIME)/1000, 2) AS avg_queue_sec,
    ROUND(SUM(BYTES_SPILLED_TO_REMOTE_STORAGE)/1073741824, 3) AS gb_spilled_remote
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE START_TIME >= DATEADD('day', -7, CURRENT_TIMESTAMP())
    AND WAREHOUSE_NAME IS NOT NULL
GROUP BY 1, 2
ORDER BY 2 DESC, 3 DESC;

-- Peak hours analysis (when is each warehouse busiest?)
SELECT
    WAREHOUSE_NAME,
    HOUR(START_TIME) AS hour_of_day,
    COUNT(*) AS query_count,
    ROUND(AVG(TOTAL_ELAPSED_TIME)/1000, 2) AS avg_sec,
    SUM(CASE WHEN TRANSACTION_BLOCKED_TIME > 0 THEN 1 ELSE 0 END) AS queued
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE START_TIME >= DATEADD('day', -30, CURRENT_TIMESTAMP())
GROUP BY 1, 2
ORDER BY 1, 2;

-- Warehouse credit cost per hour over last 24 hours
SELECT
    WAREHOUSE_NAME,
    DATE_TRUNC('hour', START_TIME) AS hour,
```

```

        ROUND(SUM(CREDITS_USED), 4)    AS credits,
        ROUND(SUM(CREDITS_USED_COMPUTE), 4) AS compute_credits,
        ROUND(SUM(CREDITS_USED_CLOUD_SERVICES), 4) AS cloud_svc_credits
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE START_TIME >= DATEADD('day', -1, CURRENT_TIMESTAMP())
GROUP BY 1, 2
ORDER BY 2 DESC, 3 DESC;

-- =====
-- PART 2: Query Acceleration Service
-- =====

USE ROLE SYSADMIN;

-- Enable QAS on the BI warehouse
ALTER WAREHOUSE bi_warehouse SET
    ENABLE_QUERY_ACCELERATION = TRUE
    QUERY_ACCELERATION_MAX_SCALE_FACTOR = 10;  -- Use up to 10x warehouse size in share

-- Check QAS settings
SHOW WAREHOUSES LIKE 'BI_WAREHOUSE';
-- Look for: enable_query_acceleration, query_acceleration_max_scale_factor columns

-- Test with a complex analytical query that might benefit from QAS
USE ROLE SYSADMIN;
USE DATABASE PERF_DEMO;
USE SCHEMA QUERY_PROFILE_LAB;
USE WAREHOUSE bi_warehouse;

-- Ad-hoc analytical query – good QAS candidate
SELECT
    s.region,
    DATE_TRUNC('month', s.sale_date) AS month,
    p.category,
    COUNT(DISTINCT s.customer_id)    AS unique_customers,
    SUM(s.amount)                    AS total_revenue,
    AVG(s.amount)                    AS avg_order_value,
    MAX(s.amount)                    AS max_order,
    RANK() OVER (
        PARTITION BY s.region, DATE_TRUNC('month', s.sale_date)
        ORDER BY SUM(s.amount) DESC
    ) AS category_rank
FROM sales_fact s
JOIN product_dim p ON s.product_id = p.product_id
WHERE s.sale_date BETWEEN DATEADD('month', -6, CURRENT_DATE()) AND CURRENT_DATE()
GROUP BY s.region, month, p.category
HAVING SUM(s.amount) > 5000
ORDER BY s.region, month, category_rank;

-- Monitor QAS usage after the query
USE ROLE ACCOUNTADMIN;
SELECT
    QUERY_ID,
    WAREHOUSE_NAME,
    LEFT(QUERY_TEXT, 80)          AS preview,
    TOTAL_ELAPSED_TIME/1000      AS elapsed_sec,
    QAS_CREDITS_USED

```

```

FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE QAS_CREDITS_USED > 0
    AND START_TIME >= DATEADD('hour', -1, CURRENT_TIMESTAMP())
ORDER BY QAS_CREDITS_USED DESC
LIMIT 10;

-- Disable QAS when done
ALTER WAREHOUSE bi_warehouse SET ENABLE_QUERY_ACCELERATION = FALSE;

-- =====
-- PART 3: Scale Up vs Scale Out Decision
-- =====
-- Check if a warehouse has queuing issues (Scale Out needed?)
SELECT
    WAREHOUSE_NAME,
    COUNT(*) AS queued_queries,
    AVG(TRANSACTION_BLOCKED_TIME)/1000 AS avg_queue_sec,
    MAX(TRANSACTION_BLOCKED_TIME)/1000 AS max_queue_sec
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE START_TIME >= DATEADD('day', -7, CURRENT_TIMESTAMP())
    AND TRANSACTION_BLOCKED_TIME > 0
GROUP BY WAREHOUSE_NAME
ORDER BY avg_queue_sec DESC;
-- If queued_queries is high and avg_queue_sec > 5, consider multi-cluster (Scale Out)

-- Check if a warehouse has spilling issues (Scale Up needed?)
SELECT
    WAREHOUSE_NAME,
    COUNT(*) AS queries_with_spill,
    ROUND(AVG(BYTES_SPILLED_TO_REMOTE_STORAGE)/1073741824, 3) AS avg_gb_spill_remote
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE START_TIME >= DATEADD('day', -7, CURRENT_TIMESTAMP())
    AND BYTES_SPILLED_TO_REMOTE_STORAGE > 0
GROUP BY WAREHOUSE_NAME
ORDER BY avg_gb_spill_remote DESC;
-- If queries spill to remote storage, Scale UP the warehouse

-- Cleanup
DROP WAREHOUSE IF EXISTS bi_warehouse;
DROP WAREHOUSE IF EXISTS ml_training_warehouse;

```

☒ Topic 3.3 — 20 Practice Questions: Warehouse Performance Tools

Q1. An analyst reports that their queries on the BI warehouse run in 2 seconds individually, but they often wait 90 seconds in the queue before execution begins. What is the correct solution?

- A) Scale UP the warehouse from Medium to Large
- B) Enable Query Acceleration Service (QAS) on the warehouse
- C) Enable multi-cluster (Scale Out) with more clusters to reduce queuing
- D) Add a clustering key to all frequently queried tables

Q2. A nightly ETL job processes 2TB of data with complex multi-table joins. It takes 6 hours on a Medium warehouse. The queries don't queue — it's always just one query running. What should be done?

- A) Enable multi-cluster warehouse to distribute the ETL across clusters
- B) Scale UP the warehouse to a larger size (e.g., Large or X-Large)
- C) Enable Query Acceleration Service
- D) Create a separate warehouse for each table in the ETL pipeline

Q3. The Query Acceleration Service (QAS) offloads query work to which type of compute?

- A) The warehouse's local SSD cache
- B) A dedicated enterprise-level virtual warehouse you provision separately
- C) Snowflake's pool of shared serverless compute resources
- D) Temporary external cloud VMs that Snowflake provisions automatically

Q4. What does `QUERY_ACCELERATION_MAX_SCALE_FACTOR = 8` mean on a warehouse?

- A) QAS can run at most 8 queries simultaneously on the warehouse
- B) QAS can use up to 8× the warehouse's own compute capacity in additional shared serverless resources
- C) QAS activates only when a query has been running for more than 8 seconds
- D) The warehouse can scale to a maximum of 8 clusters using QAS

Q5. Which of the following scenarios is the BEST candidate for the Query Acceleration Service?

- A) 500 concurrent dashboard users all running a standard 3-second report
- B) A weekly ad-hoc analytics query that unpredictably scans billions of rows with highly selective filters
- C) A nightly ETL job that predictably loads 1 million rows from S3 every night
- D) A simple `SELECT * FROM small_table` that runs in 1 second

Q6. Search Optimization Service (SOS) and Query Acceleration Service (QAS) are used together on the same query. In what order do they operate?

- A) QAS runs first to accelerate the scan, then SOS verifies the results
- B) SOS runs first to prune irrelevant micro-partitions, then QAS accelerates the scan of remaining partitions
- C) They run simultaneously — SOS handles columns, QAS handles rows
- D) Only one can be active per query — Snowflake chooses the more effective one

Q7. Which `ACCOUNT_USAGE` column would you check to determine if queries were waiting in a warehouse queue?

- A) EXECUTION_TIME — total execution duration
- B) COMPILATION_TIME — query parse and compile time
- C) TRANSACTION_BLOCKED_TIME — time the query spent waiting in the warehouse queue
- D) BYTES_SCANNED — data volume read

Q8. A warehouse has `ENABLE_QUERY_ACCELERATION = TRUE` but `QUERY_ACCELERATION_MAX_SCALE_FACTOR = 0`. What happens?

- A) QAS is fully active with unlimited scaling
- B) QAS is effectively disabled — a scale factor of 0 prevents any serverless acceleration
- C) QAS can only accelerate queries that scan 0 partitions (metadata-only)
- D) The setting is invalid and Snowflake ignores it

Q9. What does the `WAREHOUSE_METERING_HISTORY` `ACCOUNT_USAGE` view track?

- A) The query history including SQL text and execution times for each warehouse
- B) The number of virtual warehouses created each day
- C) Hourly credit consumption per warehouse, broken down by compute and cloud services credits
- D) The storage consumption per warehouse cluster

Q10. A developer notices that their Medium warehouse has `BYTES_SPILLED_TO_REMOTE_STORAGE > 0` for 30% of all queries. What does this indicate?

- A) The queries are reading from external tables in cloud storage — expected behavior
- B) 30% of queries are exceeding available memory AND local SSD, spilling intermediate results to cloud storage — the warehouse is undersized for these queries
- C) 30% of queries are using the warehouse cache efficiently
- D) The warehouse is over-provisioned and wasting credits on unused memory

Q11. Which is the correct approach when a SINGLE complex query is slow (not a concurrency issue)?

- A) Add more clusters (Scale Out) to distribute the query across multiple clusters
- B) Scale UP the warehouse size — more memory and compute per node speeds up a single complex query
- C) Enable multi-cluster Maximized mode to guarantee all clusters are available
- D) Reduce `MAX_CONCURRENCY_LEVEL` to give the query more exclusive warehouse resources

Q12. QAS does NOT help which of the following query types? (*Select ALL that apply*)

- A) Ad-hoc analytical query scanning 10 billion rows with a selective WHERE clause
- B) Simple `INSERT INTO table VALUES (...)` DML operation

- C) A query running in under 1 second on a well-cached warehouse
- D) A complex window function query on a 5TB fact table
- E) `SELECT COUNT(*) FROM table` served from the Metadata Cache

Q13. Where in Snowsight can you visually monitor warehouse usage, including query counts and concurrency over time?

- A) Data → Databases tab
- B) Admin → Warehouses → click the warehouse name → usage graphs
- C) Activity → Query Profile → warehouse summary
- D) Governance → Trust Center → warehouse metrics

Q14. A warehouse has `AUTO_SUSPEND = 120`. During a peak period, a complex query runs for 20 minutes. After the query finishes, when does the warehouse suspend?

- A) Immediately — `AUTO_SUSPEND` always takes effect the moment a query ends
- B) 120 seconds (2 minutes) after the last query activity ends
- C) The warehouse suspends during the query at the 120-second mark
- D) The warehouse never suspends during the 20-minute query — only after 120 seconds of inactivity post-query

Q15. QAS uses which billing model?

- A) Charged against the assigned virtual warehouse credits
- B) Included free with Enterprise edition — no additional charge
- C) Separate serverless compute credits — independent of warehouse credits
- D) Charged as cloud storage fees per GB of data processed

Q16. Which of the following is NOT a valid `ACCOUNT_USAGE` view for monitoring warehouse performance?

- A) `WAREHOUSE_METERING_HISTORY`
- B) `QUERY_HISTORY`
- C) `WAREHOUSE_LOAD_HISTORY`
- D) `TABLE_STORAGE_METRICS`

Q17. A single-cluster warehouse receives 20 simultaneous queries. `MAX_CONCURRENCY_LEVEL = 8`. What happens to the remaining 12 queries?

- A) They are rejected with an error message
- B) They are queued and wait until running queries complete and slots become available
- C) They automatically spawn additional clusters to handle the extra load
- D) They run on other available warehouses in the account automatically

Q18. Which metric in Query History helps identify "idle warehouse time" (credits spent with no queries running)?

- A) `BYTES_SCANNED = 0` indicates idle time
- B) `CREDITS_USED_COMPUTE - CREDITS_ATTRIBUTED_COMPUTE_QUERIES` in `WAREHOUSE_METERING_HISTORY` gives idle compute credits
- C) `TOTAL_ELAPSED_TIME = 0` indicates the warehouse was idle
- D) `PARTITIONS_SCANNED = 0` indicates idle time

Q19. Which of the following are valid reasons to use SEPARATE dedicated warehouses for different workloads? (*Select ALL that apply*)

- A) To prevent a heavy ETL job from slowing down simultaneous BI dashboard queries
- B) To enable different scaling, auto-suspend, and resource monitor settings per workload type
- C) To avoid sharing warehouse cache — each workload has its own warm cache
- D) Because Snowflake requires a dedicated warehouse per schema
- E) To enable workload isolation — one workload's credit usage doesn't impact another

Q20. A QAS-accelerated query uses 5 credits from the warehouse AND 3 credits from QAS serverless compute. How does billing work?

- A) Only the 5 warehouse credits are billed — QAS is included in the warehouse cost
- B) Both are billed separately: 5 warehouse credits from the virtual warehouse budget, and 3 QAS credits from the serverless compute pool
- C) The total 8 credits are billed as warehouse credits only
- D) Only the higher of the two (5 warehouse credits) is billed

SUBDOMAIN 3.4 — Optimize Query Performance

☒ Layman's Explanation — The Four Optimization Tools

Clustering Keys: Imagine a library where books are shelved in random order. When you want all books from 2025, you have to scan every shelf. A clustering key is like re-shelving all books in chronological order — now you only open the 2025 section.

Materialized Views: Instead of re-running an expensive calculation every time someone asks for last month's sales by region, you pre-compute it once and store the answer in a special pre-built report. Next time someone asks, you just hand them the pre-built report.

Search Optimization Service (SOS): Like a library card catalog (index). When someone needs ONE specific book by its ISBN, instead of scanning every shelf, they look up the ISBN in the catalog and go directly to that shelf. SOS builds a persistent index for lookups.

Result Cache / Persisted Query Results: If 100 users ask the same question today, Snowflake computes the answer ONCE, stores it for 24 hours, and serves all 100 users the cached answer instantly — zero re-computation.

☒ Technical Explanation

Clustering Keys — Deep Dive^{[16][17][^18]}

What is **natural clustering**? When you load data into Snowflake, rows are stored in micro-partitions in the ORDER they were inserted. If you load sales data chronologically, `sale_date` will naturally cluster well. But after many random inserts, updates, or merges, the clustering degrades.

Clustering Depth and Overlap:^{[19][20][^17]}

- **Clustering Depth:** Average number of micro-partitions that overlap for a given value of the clustering key column. Depth = 1 means perfect clustering (each value in only one partition). High depth = poor clustering.
- **Average Overlaps:** Average number of overlapping micro-partitions per partition.
- Lower depth = better pruning efficiency = faster queries.

```
-- Check clustering information
SELECT SYSTEM$CLUSTERING_INFORMATION('my_table', '(sale_date)');
-- Returns: total_partition_count, average_overlaps, average_depth, partition_depth.

SELECT SYSTEM$CLUSTERING_DEPTH('my_table', '(sale_date)');
-- Returns: just the average depth number

SELECT SYSTEM$ESTIMATE_AUTOMATIC_CLUSTERING_COSTS('my_table', '(sale_date)');
-- Estimates the ongoing cost of automatic clustering
```

Key rules for choosing clustering keys:^{[16][4]}

- Use columns frequently in WHERE and JOIN conditions
- Good candidates: date/timestamp columns, region/category (low-to-medium cardinality)
- Bad candidates: high-cardinality columns (UUID, email — too many unique values = poor pruning)
- Bad candidates: frequently updated columns (updates cause constant re-clustering overhead)
- Maximum ONE clustering key per table (but can include multiple columns in the key)
- Can use expressions: `CLUSTER BY (DATE_TRUNC('month', order_date))`

Automatic Clustering (AC):^{[20][18]}

- After adding a clustering key, Snowflake maintains it in the background automatically
- AC is a **serverless, paid feature** — charges appear in `AUTOMATIC_CLUSTERING_HISTORY`

- To disable AC while keeping the key definition: `ALTER TABLE t SUSPEND RECLUSTER`
- To re-enable: `ALTER TABLE t RESUME RECLUSTER`

Materialized Views^{[21][22][15][13]}

What they are: Pre-computed, stored result sets derived from a query. Snowflake automatically maintains them when source data changes (serverless background refresh).

SQL syntax:

```
CREATE MATERIALIZED VIEW monthly_sales_mv
  CLUSTER BY (region, sale_month)  -- Can have its own clustering key!
AS
  SELECT
    region,
    DATE_TRUNC('month', sale_date) AS sale_month,
    SUM(amount) AS total_revenue,
    COUNT(*) AS num_orders
  FROM sales_fact
  GROUP BY region, sale_month;
```

Key behaviors and edge cases:^{[22][15][^13]}

- The materialized view is **automatically refreshed** by Snowflake when source table data changes — serverless credits consumed
- Queries against the materialized view return pre-computed results — very fast
- Snowflake can also **transparently rewrite** a query against the base table to use the materialized view if the optimizer determines it's faster — this is called **query rewrite**
- Materialized views have their **OWN** clustering key — independent of the source table's clustering key
- Can store flattened VARIANT/JSON data — avoiding repeated FLATTEN overhead

Materialized View limitations (critical exam traps):^[^22]

- Source table **CANNOT** have a Column-Level Security (masking) policy — if DDM is applied, cannot create MV
- NOT supported on: External Tables, Dynamic Tables, Views with non-deterministic functions (CURRENT_TIMESTAMP, etc.)
- Only **ONE** materialized view's results can be used per query rewrite per table
- Requires **Enterprise edition**
- Subqueries in the MV definition: **NOT** allowed (no nested SELECT... FROM SELECT...)

Search Optimization Service (SOS)^{[23][15][^13]}

What it builds: A persistent search access path — a server-side index that maps specific column values to the micro-partitions containing them. Queries using equality or range filters on these columns can jump directly to the relevant partitions instead of scanning all partitions.

Supported search types:^{[15][23]}

- EQUALITY — exact match: `WHERE customer_id = 12345`
- RANGE — range queries: `WHERE sale_date BETWEEN '2025-01-01' AND '2025-03-31'`
- SUBSTRING — LIKE and CONTAINS: `WHERE email LIKE '%@gmail.com'`
- REGEX — regular expression matching
- GEO — geospatial point-in-polygon, distance queries on GEOGRAPHY columns

```
-- Enable SOS on entire table (all supported columns)
ALTER TABLE large_orders ADD SEARCH OPTIMIZATION;

-- Enable SOS on specific columns only (more cost-efficient)
ALTER TABLE large_orders ADD SEARCH OPTIMIZATION ON
    EQUALITY(customer_id, product_id),
    SUBSTRING(email),
    RANGE(sale_date);

-- Remove SOS
ALTER TABLE large_orders DROP SEARCH OPTIMIZATION;

-- Check SOS status and cost
SELECT SYSTEM$SEARCH_OPTIMIZATION_STATISTICS('MY_DB.MY_SCHEMA.LARGE_ORDERS');
```

Cost: Storage cost (for the search access path) + serverless compute to build and maintain it.
Requires: Enterprise edition.

Result Cache / Persisted Query Results^{[24][25][26][27]}

Duration: 24 hours, extended by an additional 24 hours each time the same query runs within the active window (up to 31 days maximum).

Conditions that INVALIDATE the result cache:

- DML on underlying tables (INSERT, UPDATE, DELETE, MERGE, TRUNCATE, COPY INTO)
- DDL on underlying tables (ALTER TABLE, DROP TABLE)
- The 24-hour window expires without the query being re-run
- Session parameter changes that affect query results
- Query uses non-deterministic functions

How to disable/check:

```
ALTER SESSION SET USE_CACHED_RESULT = FALSE;    -- Disable for testing
ALTER SESSION SET USE_CACHED_RESULT = TRUE;     -- Re-enable (default)
SHOW PARAMETERS LIKE 'USE_CACHED_RESULT';       -- Check current setting
```

Result Cache is account-wide: If User A runs a query at 9 AM, and User B (with the same access permissions) runs the identical query at 10 AM — User B gets the cached result from User A's run.

SELECT Command Optimization Tips^[28]_[4]

Technique	Why
Only SELECT needed columns	Fewer columns = fewer bytes read from micro-partitions (columnar storage benefit)
Add WHERE clauses	Reduces data scanned — enables micro-partition pruning
Use LIMIT for exploration	Stops execution after N rows — good for previewing data without full scan
Avoid SELECT DISTINCT on large tables	DISTINCT requires a full deduplication pass — expensive on billions of rows
Filter early in CTEs	Push filters as close to the data source as possible
Avoid functions on clustered columns in WHERE	WHERE TO_CHAR(sale_date) = '2025-01-15' breaks pruning — use WHERE sale_date = '2025-01-15'

☒ Hands-On Queries: Optimization Techniques

```
-- =====
-- SETUP
-- =====

USE ROLE SYSADMIN;
USE DATABASE PERF_DEMO;
CREATE SCHEMA IF NOT EXISTS OPTIMIZATION_LAB;
USE SCHEMA OPTIMIZATION_LAB;
USE WAREHOUSE learn_wh;

CREATE OR REPLACE TABLE big_orders (
  order_id    INTEGER,
  order_date  DATE,
  region      VARCHAR(30),
  category    VARCHAR(50),
  customer_id INTEGER,
  amount      NUMBER(12,2)
);

INSERT INTO big_orders
SELECT
  SEQ4()                                AS order_id,
  DATEADD('day', -MOD(SEQ4(), 365), CURRENT_DATE()) AS order_date,
  CASE MOD(SEQ4(), 4)
```

```

        WHEN 0 THEN 'North' WHEN 1 THEN 'South'
        WHEN 2 THEN 'East' ELSE 'West' END          AS region,
CASE MOD(SEQ4(), 5)
    WHEN 0 THEN 'Electronics' WHEN 1 THEN 'Clothing'
    WHEN 2 THEN 'Food' WHEN 3 THEN 'Tools'
    ELSE 'Books' END                                AS category,
MOD(SEQ4(), 100000)                                AS customer_id,
ROUND(UNIFORM(5.0, 5000.0, RANDOM()), 2)           AS amount
FROM TABLE(GENERATOR(ROWCOUNT => 500000));

```

```

-- =====
-- PART 1: Clustering Keys
-- =====

```

```

ALTER SESSION SET USE_CACHED_RESULT = FALSE;

```

```

-- Check clustering before adding key
SELECT SYSTEM$CLUSTERING_INFORMATION(
    'PERF_DEMO.OPTIMIZATION_LAB.BIG_ORDERS', '(order_date)'
) AS clustering_info;
-- Note: average_depth and average_overlaps (high = poor clustering)

```

```

-- Query without clustering (full scan)
SELECT SUM(amount), COUNT(*) FROM big_orders
WHERE order_date = '2025-12-15';
-- Query Profile: Partitions Scanned ≈ Partitions Total

```

```

-- Add clustering key
ALTER TABLE big_orders CLUSTER BY (order_date);

```

```

-- Re-check clustering info (background AC will improve over time)
SELECT SYSTEM$CLUSTERING_INFORMATION(
    'PERF_DEMO.OPTIMIZATION_LAB.BIG_ORDERS', '(order_date)'
);
-- After AC runs: average_depth decreases

```

```

-- Check clustering depth only (quick check)
SELECT SYSTEM$CLUSTERING_DEPTH(
    'PERF_DEMO.OPTIMIZATION_LAB.BIG_ORDERS', '(order_date)'
) AS avg_depth;

```

```

-- After clustering settles, same query shows better pruning
SELECT SUM(amount), COUNT(*) FROM big_orders
WHERE order_date = '2025-12-15';

```

```

-- Multi-column clustering key (date + region)
ALTER TABLE big_orders CLUSTER BY (order_date, region);

```

```

-- Compound filter benefits from multi-column clustering
SELECT SUM(amount), COUNT(*) FROM big_orders
WHERE order_date = '2025-12-15' AND region = 'North';

```

```

-- =====
-- PART 2: Materialized Views
-- =====

```

```

-- Create a materialized view for frequently-run aggregation
CREATE OR REPLACE MATERIALIZED VIEW monthly_category_mv

```

```

COMMENT = 'Pre-aggregated monthly revenue by category and region'
AS
SELECT
    region,
    category,
    DATE_TRUNC('month', order_date) AS order_month,
    COUNT(*)                        AS num_orders,
    SUM(amount)                     AS total_revenue,
    AVG(amount)                     AS avg_order_value
FROM big_orders
GROUP BY region, category, order_month;

-- Query the materialized view – very fast!
SELECT * FROM monthly_category_mv
WHERE region = 'North'
ORDER BY order_month DESC, total_revenue DESC;

-- The materialized view may also be used for query rewrite:
-- If you query the base table with the same aggregation, Snowflake MAY
-- transparently rewrite to use the MV internally
SELECT region, category,
    DATE_TRUNC('month', order_date) AS month,
    COUNT(*), SUM(amount)
FROM big_orders
WHERE region = 'North'
GROUP BY region, category, month;
-- Check Query Profile: if "TableScan on MONTHLY_CATEGORY_MV" appears → query rewrite

-- Monitor MV maintenance
SHOW MATERIALIZED VIEWS IN SCHEMA OPTIMIZATION_LAB;
-- Key columns: is_secure, cluster_by, rows, bytes, invalid_reason, behind_by

-- =====
-- PART 3: Search Optimization Service
-- =====
-- Add Search Optimization for equality lookups on customer_id
ALTER TABLE big_orders ADD SEARCH OPTIMIZATION ON
    EQUALITY(customer_id),
    EQUALITY(order_id);

-- Check SOS status (takes a few minutes to build the access path)
SELECT SYSTEM$SEARCH_OPTIMIZATION_STATISTICS(
    'PERF_DEMO.OPTIMIZATION_LAB.BIG_ORDERS'
);
-- Shows: STATUS, indexed columns, storage cost

-- Point lookup WITHOUT SOS (full scan)
-- ALTER TABLE big_orders DROP SEARCH OPTIMIZATION; -- uncomment to compare
SELECT * FROM big_orders WHERE customer_id = 54321;

-- Point lookup WITH SOS (uses search access path)
SELECT * FROM big_orders WHERE customer_id = 54321;
-- Query Profile: should show fewer partitions scanned!

-- =====
-- PART 4: Result Cache Behavior

```

```

-- =====
ALTER SESSION SET USE_CACHED_RESULT = TRUE;

-- First run (populates cache)
SELECT region, category, SUM(amount)
FROM big_orders
GROUP BY region, category
ORDER BY region, category;

-- Second run (served from cache)
SELECT region, category, SUM(amount)
FROM big_orders
GROUP BY region, category
ORDER BY region, category;
-- Query Profile: QUERY RESULT REUSE

-- Invalidate cache by DML
INSERT INTO big_orders VALUES (999999, CURRENT_DATE(), 'North', 'Electronics', 99999,

-- Now re-run: cache is INVALIDATED (data changed), full scan happens
SELECT region, category, SUM(amount)
FROM big_orders
GROUP BY region, category
ORDER BY region, category;

-- Check session parameter
SHOW PARAMETERS LIKE 'USE_CACHED_RESULT' IN SESSION;

-- =====
-- PART 5: SELECT Query Optimization
-- =====
-- Bad: SELECT * (reads all columns unnecessarily)
-- SELECT * FROM big_orders WHERE order_date = CURRENT_DATE() - 30;

-- Better: Only select needed columns (fewer bytes read)
SELECT order_id, customer_id, amount
FROM big_orders
WHERE order_date = CURRENT_DATE() - 30;

-- Bad: Function on clustered column breaks pruning
SELECT * FROM big_orders
WHERE TO_CHAR(order_date, 'YYYY-MM-DD') = TO_CHAR(CURRENT_DATE() - 30, 'YYYY-MM-DD');
-- Query Profile: full scan! TO_CHAR() prevents partition pruning

-- Good: Direct date comparison allows pruning
SELECT * FROM big_orders
WHERE order_date = CURRENT_DATE() - 30;
-- Query Profile: much better pruning!

-- Clean up
DROP TABLE IF EXISTS big_orders;
DROP MATERIALIZED VIEW IF EXISTS monthly_category_mv;
DROP DATABASE IF EXISTS PERF_DEMO;

```

☒ Topic 3.4 — 20 Practice Questions: Query Optimization

Q1. A table has a clustering key defined on `(order_date)`. Which query would BEST benefit from partition pruning?

- A) `SELECT COUNT(*) FROM orders` — no filter
- B) `SELECT * FROM orders WHERE TO_CHAR(order_date, 'YYYY') = '2025'` — year function
- C) `SELECT * FROM orders WHERE order_date BETWEEN '2025-01-01' AND '2025-01-31'` — direct range filter
- D) `SELECT * FROM orders ORDER BY amount DESC` — no date filter

Q2. What does `SYSTEM$CLUSTERING_INFORMATION` return for a table?

- A) The query execution plan for the most recent query on the table
- B) Clustering statistics including total partition count, average overlap depth, and a partition depth histogram
- C) The list of all clustering keys defined on the table
- D) The credit cost of automatic clustering for the past 30 days

Q3. A Materialized View is defined on `sales_fact`. New rows are inserted into `sales_fact` every hour. How is the Materialized View kept current?

- A) The MV must be manually refreshed by running `REFRESH MATERIALIZED VIEW`
- B) Snowflake automatically maintains the MV in the background using serverless compute — refreshing it when source data changes
- C) The MV is rebuilt from scratch every 24 hours on a schedule
- D) The MV is only refreshed when someone queries it — it's lazy-evaluated

Q4. Which of the following prevents creating a Materialized View on a base table?

- A) The base table has a clustering key defined
- B) The base table has a column with a Dynamic Data Masking policy applied
- C) The base table has more than 100 columns
- D) The base table is stored in a schema with `TRANSIENT` retention

Q5. Which Snowflake Search Optimization type would help speed up `SELECT * FROM customers WHERE email LIKE '%@gmail.com'?`

- A) `EQUALITY(email)` — exact match search
- B) `RANGE(email)` — range-based search
- C) `SUBSTRING(email)` — substring search (`LIKE`, `CONTAINS`, `STARTSWITH`)
- D) `GEO(email)` — geospatial search

Q6. What does "clustering depth = 1" mean for a table's clustering key?

- A) The clustering key has only 1 column
- B) The table only has 1 micro-partition
- C) Perfect clustering — each distinct value of the clustering key exists in only 1 micro-partition (no overlap)
- D) The clustering key was just created and hasn't been applied yet

Q7. What is Automatic Clustering (AC) in Snowflake?

- A) Snowflake's automatic choice of the best clustering key for a table
- B) A background serverless process that continuously re-sorts and reorganizes micro-partitions to maintain the clustering key
- C) A query optimization that reorders query steps to improve partition pruning
- D) An automatic data compression feature that runs weekly

Q8. User A runs a complex query at 9 AM. The Result Cache stores the result. User B, with identical access permissions, runs the EXACT same query at 11 AM (no data changes). What does User B experience?

- A) User B must wait for a full re-computation because Result Cache is user-specific
- B) User B gets the cached result instantly — Result Cache is account-wide and shared across all users with the same permissions
- C) User B gets the result from the Warehouse Local Cache of their warehouse
- D) User B gets a stale result — Result Cache only serves the user who created it

Q9. Which SELECT pattern breaks micro-partition pruning on a clustered column? (*Select ALL that apply*)

- A) `WHERE sale_date = '2025-01-15'` — direct equality
- B) `WHERE TO_CHAR(sale_date, 'YYYY-MM-DD') = '2025-01-15'` — function wraps the column
- C) `WHERE YEAR(sale_date) = 2025` — function extracts year
- D) `WHERE sale_date BETWEEN '2025-01-01' AND '2025-12-31'` — direct range
- E) `WHERE DATE_TRUNC('month', sale_date) = '2025-01-01'` — expression on column

Q10. Materialized Views are useful for which of the following use cases? (*Select ALL that apply*)

- A) Pre-aggregating large tables for frequent analytical queries
- B) Defining a DIFFERENT clustering key on the same source table data
- C) Flattening semi-structured (VARIANT) data so it only needs to be processed once
- D) Storing raw data backups that are refreshed every night
- E) Speeding up frequently-run queries on subsets of source table rows and columns

Q11. What happens to the Result Cache when an UPDATE statement is run on the queried table?

- A) Only cached queries that touched the updated rows are invalidated
- B) The Result Cache for ALL queries on that table is invalidated — any DML on the table clears related cache entries
- C) The cache remains valid — updates don't change the query result for rows that weren't updated
- D) The cache is invalidated only if more than 10% of rows were updated

Q12. A table `orders` is clustered on `(region)`. A query filters `WHERE order_date = '2025-01-15'`. What kind of pruning occurs?

- A) Excellent pruning — clustering on `region` covers all column filters
- B) No pruning benefit for the `order_date` filter — the clustering key is `region`, not `order_date`. Snowflake must scan all partitions to find the date.
- C) Snowflake automatically adds a secondary cluster key for `order_date`
- D) The query uses the search optimization path for `order_date` automatically

Q13. The `USE_CACHED_RESULT = FALSE` session parameter is set. A user runs a query. What happens?

- A) The query fails with an error — caching cannot be disabled
- B) The Result Cache is bypassed — Snowflake always re-executes the query against the source data
- C) The Warehouse Local Cache is also disabled
- D) Only the Metadata Cache is disabled; Result Cache still applies

Q14. What is the MAXIMUM duration the Result Cache can hold a result (if the query is repeatedly re-run within each 24-hour window)?

- A) 24 hours
- B) 7 days
- C) 31 days
- D) 90 days

Q15. A developer wants to estimate how much Automatic Clustering will cost BEFORE enabling it. Which Snowflake function should they use?

- A) `SYSTEM$CLUSTERING_DEPTH('table', '(col)')`
- B) `SYSTEM$CLUSTERING_INFORMATION('table', '(col)')`
- C) `SYSTEM$ESTIMATE_AUTOMATIC_CLUSTERING_COSTS('table', '(col)')`
- D) `SYSTEM$CLUSTERING_COST_ESTIMATE('table')`

Q16. Search Optimization Service and Clustering Keys both improve query performance. What is the KEY difference between them?

- A) Clustering Keys work on large range scans; SOS works on point lookups and substring searches
- B) Clustering Keys require Enterprise edition; SOS works on all editions
- C) SOS is free; Clustering Keys cost credits
- D) SOS works on all query types; Clustering Keys only work on date columns

Q17. Which of the following are costs associated with Materialized Views? (*Select ALL that apply*)

- A) Storage cost — the pre-computed result set uses physical storage
- B) Serverless compute cost — Snowflake uses serverless credits to maintain/refresh the MV
- C) Warehouse compute cost — each refresh runs on the specified virtual warehouse
- D) Additional storage cost for clustering if the MV has a clustering key
- E) Query cost — each query of the MV runs on the specified warehouse

Q18. A `SELECT * FROM orders` query returns 500 million rows in 45 seconds. The same query with `SELECT order_id, amount FROM orders` returns in 30 seconds. Why?

- A) The narrower SELECT uses a different code path in Snowflake's optimizer
- B) Snowflake's columnar storage means fewer columns = fewer bytes read from micro-partitions, reducing scan time
- C) `SELECT *` triggers Result Cache invalidation; the narrower SELECT does not
- D) `SELECT *` requires full table locks; the narrower SELECT does not

Q19. A table has NO clustering key defined. `SYSTEM$CLUSTERING_DEPTH` returns a very high value (e.g., 500). What does this indicate?

- A) The table is perfectly optimized — depth of 500 means 500 distinct partitions
- B) The table has very poor physical ordering — each distinct value spans approximately 500 micro-partitions, making pruning nearly impossible
- C) The table needs to be split into 500 smaller tables
- D) The table has 500 clustering keys competing with each other

Q20. Snowflake can transparently rewrite a query against a base table to instead use a Materialized View. What is this feature called?

- A) Query Acceleration — QAS routes queries to the MV
- B) Automatic Query Rewrite — Snowflake's optimizer detects that the MV can answer the query and uses it
- C) Dynamic View Resolution — Snowflake resolves views dynamically at runtime
- D) Partition Substitution — Snowflake replaces scanned partitions with the MV's cached partitions

☒ COMPLETE ANSWER KEY — Domain 3: Performance Optimization

Topic 3.1 — Query Profile

Q	Answer	Key Reason
1	B	Partitions Scanned (15) << Partitions Total (12,400) = excellent pruning. Clustering key is working. [^3]
2	C	"QUERY RESULT REUSE" node = Result Cache hit. The complete result was served from the Cloud Services cache. [24][26]
3	B	Bytes Spilled to Remote Storage = intermediate results exceeded memory AND local SSD → went to cloud storage. Very severe performance degradation. [3][6]
4	C	Exploding Join / Join Explosion — the join output row count vastly exceeds the input row counts, indicating a many-to-many relationship or missing join condition. [5][7]
5	B, C	QUERY RESULT REUSE (Result Cache) ✓ and METADATA-BASED RESULT (Metadata Cache) ✓ both answer queries without any warehouse compute. TableScan, Filter, Aggregate all require a warehouse. [26][24]
6	B	SELECT COUNT (*) with no WHERE is served from the Metadata Cache — row counts are stored in Cloud Services metadata. Profile shows "METADATA-BASED RESULT". [24][29]
7	C	Non-deterministic functions (CURRENT_TIMESTAMP, RANDOM, UUID_STRING, etc.) prevent Result Cache use — they produce different values on each call, so a cached result would be incorrect. [^26]
8	B	BROADCAST exchange = a small table is copied to all warehouse nodes so each node has it locally for the join, eliminating data transfer during join processing. [^1]
9	C	High Transaction Blocked Time = queries waiting in the warehouse queue. Fix: multi-cluster warehouse (Scale Out). [^3]
10	B	Bytes Spilled to Local Storage = overflow from in-memory buffer to warehouse SSD. Less severe than remote spill but still indicates warehouse may need scaling up. [3][6]
11	C	The Result Cache served the second run — same query text, no data changes between runs, USE_CACHED_RESULT = TRUE. Returns in milliseconds from Cloud Services layer. [24][26]
12	B, C, E	DML on the table ✓; using RANDOM() (non-deterministic) ✓; 24-hour expiration ✓. Different warehouse ✗ — Result Cache is account-wide, not warehouse-specific. Same query text ✗ — that's a HIT condition. [24][26][^27]
13	B	High Aggregate node cost = too many rows entering aggregation, possibly due to missing or late-applied filter. Push filters earlier to reduce rows before aggregation. [^4]
14	C	Snowsight Query History (and Query Profile) retains data for 14 days. [^3]
15	C	SNOWFLAKE . ACCOUNT_USAGE . QUERY_HISTORY contains BYTES_SPILLED_TO_REMOTE_STORAGE, PARTITIONS_SCANNED, PARTITIONS_TOTAL, and all other per-query performance metrics. [^4]

Q	Answer	Key Reason
16	B	WHERE clause requires actual data scanning to filter rows — metadata only stores MIN/MAX/COUNT for the entire partition, not filtered subsets. The filter breaks Metadata Cache eligibility.[^24]
17	B	Always start with the Query Profile and "Most Expensive Nodes" — diagnose before treating. Blindly scaling up is wasteful if the real problem is a missing WHERE clause or bad join.[^4]
18	B	INEFFICIENT_JOIN_CONDITION = the join condition is a complex expression evaluated AFTER the join, rather than being pushed down as a pre-join filter.[^5]
19	C	Not every full scan needs intervention. If the query is infrequent, the table is small, or full-table analytics is the intent, 100% scan is acceptable. Context matters.[^6]
20	B	WithReference/WithClause — Snowflake materializes a CTE once and references it for each subsequent use in the query plan.[^1]

Topic 3.2 — Virtual Warehouse Configurations

Q	Answer	Key Reason
1	B	60-second minimum billing applies whenever a warehouse transitions from SUSPENDED to RUNNING. Medium = 4 credits/hr. 60 sec = $60/3600 \times 4 = 0.0667$ credits.[^2]
2	B	Scale Up = larger warehouse size = more compute power per query (solves slow single queries). Scale Out = multi-cluster = more simultaneous capacity (solves concurrency queuing).[⁸][2]
3	B	300 concurrent users queuing = a concurrency problem → Scale Out with multi-cluster warehouse to add more parallel clusters.[¹⁰][11]
4	B	MIN_CLUSTER_COUNT = MAX_CLUSTER_COUNT = Maximized mode. All defined clusters always run.[¹⁰][11]
5	C	Resizing a warehouse completely clears the local SSD cache. New nodes have empty caches — first queries after resize must read from remote cloud storage.[²⁴][26]
6	D	Snowpark-Optimized warehouses provide 16× more memory per node than Standard warehouses of the same size.[^8]
7	B	Standard policy: starts a cluster when a query is queued OR when Snowflake estimates current resources are insufficient for additional queries.[^11]
8	C	Snowpark-Optimized warehouses are single-cluster only — they cannot be used in multi-cluster mode.[^8]
9	B	AUTO_RESUME = TRUE automatically resumes a suspended warehouse when a new SQL query is submitted to it.[^8]
10	C	Snowpark-Optimized Medium provides 16× more memory per node — ideal for in-memory ML training on large DataFrames.[^8]
11	B	Economy policy prioritizes saving credits: waits longer before starting new clusters (accepts more queuing) and shuts idle clusters down faster.[^11]
12	B	AUTO_SUSPEND = 60 means: suspend 60 seconds after the LAST query completes. Query finishes at 3:00 PM → suspend at 3:01 PM.[^8]

Q	Answer	Key Reason
13	B	MAX_CONCURRENCY_LEVEL = max simultaneous SQL statements per cluster. Reaching this limit: single-cluster queues; multi-cluster auto-scale starts new cluster.[^12]
14	C	Resizing a RUNNING warehouse queues the resize — it takes effect for NEW queries after in-flight queries complete. The current query finishes on the old size.[^8]
15	C	Credits = running clusters × size rate. 3 clusters × 4 credits/hr (Medium) = 12 credits/hour.[^10][^11]
16	B	STATEMENT_QUEUED_TIMEOUT_IN_SECONDS cancels a query that has been in the queue longer than the specified timeout. Query is cancelled with an error.[^12]
17	C	Maximized mode = MIN_CLUSTER_COUNT = MAX_CLUSTER_COUNT. All clusters always running regardless of load.[^10][^11]
18	B	Cloud Services credits are free up to 10% of daily compute credits. Above 10%: excess billed separately.[^2]
19	A, C, D, E	Multi-cluster solves concurrency ✓; auto-suspend applies to entire warehouse ✓; Enterprise required ✓; Maximized keeps all clusters running ✓. Does NOT solve single-query performance ✗.[^10][^11][^9]
20	B	AUTO_RESUME = TRUE: warehouse automatically resumes when a query is submitted. The user's query waits a few seconds for the resume, then executes.[^8]

Topic 3.3 — Warehouse Performance Tools

Q	Answer	Key Reason
1	C	Queries run fast (2s) but queue 90s = concurrency problem → Scale Out (multi-cluster), not Scale Up.[^2][^8]
2	B	Single slow complex query, no queuing = compute bottleneck for one query → Scale UP the warehouse to give it more memory and parallelism.[^8][^4]
3	C	QAS offloads eligible query work to Snowflake's pool of shared serverless compute resources.[^14][^13]
4	B	Scale factor 8 = QAS can use up to 8× the warehouse's own compute in shared serverless resources.[^14]
5	B	QAS is best for ad-hoc analytical queries with unpredictable data volume, large scans, and selective filters — "outlier" queries.[^13][^14]
6	B	SOS runs FIRST (prunes micro-partitions), then QAS runs on the remaining data. Complementary — both can work on the same query.[^15][^13]
7	C	TRANSACTION_BLOCKED_TIME in QUERY_HISTORY = time query spent waiting in the warehouse queue.[^4]
8	B	SCALE_FACTOR = 0 effectively disables QAS acceleration (no shared resources allocated).[^14]
9	C	WAREHOUSE_METERING_HISTORY tracks hourly credit consumption per warehouse, broken down by compute and cloud services.[^30]
10	B	BYTES_SPILLED_TO_REMOTE_STORAGE > 0 for 30% of queries = those queries exceed memory AND SSD → warehouse too small for those query types.[^3][^6]

Q	Answer	Key Reason
11	B	Single slow query = Scale UP (more memory/compute per node). Scale Out (multi-cluster) adds parallel workers for concurrency, not per-query power.[⁸][2]
12	B, C, E	DML INSERT INTO ✗ QAS doesn't help; sub-second queries on warm cache ✗; Metadata Cache queries ✗ (no scanning happens). Large analytical queries ✓; ad-hoc complex queries ✓.[¹⁴][14]
13	B	Admin → Warehouses → click warehouse name → shows usage graph, query counts, concurrency over time.[³]
14	D	AUTO_SUSPEND starts counting AFTER the query activity ends. Query runs 20 minutes → ends → 120 seconds later → auto-suspend triggers.[⁸]
15	C	QAS uses separate serverless compute credits — independent of the warehouse credits.[¹⁴][13]
16	D	TABLE_STORAGE_METRICS is a storage monitoring view, NOT a warehouse performance monitoring view. The others are all valid warehouse performance views.[¹]
17	B	When MAX_CONCURRENCY_LEVEL is reached on a single-cluster warehouse, excess queries queue. They wait for a running query to complete and free a slot.[¹²]
18	B	Idle credits = CREDITS_USED_COMPUTE - CREDITS_ATTRIBUTED_COMPUTE_QUERIES in WAREHOUSE_METERING_HISTORY.[³¹]
19	A, B, C, E	Workload isolation ✓; different settings ✓; own warm cache ✓; credit isolation ✓. Snowflake does NOT require dedicated warehouses per schema ✗.[⁹]
20	B	Both warehouse credits (5) and QAS serverless credits (3) are billed separately — each from their respective billing pools.[¹⁴]

Topic 3.4 — Query Optimization

Q	Answer	Key Reason	
1	C	Direct range filter on the clustered column order_date = best pruning. Functions on the column (TO_CHAR) break pruning; no filter = full scan.[¹⁶][4]	
2	B	SYSTEM\$CLUSTERING_INFORMATION returns partition count, average_overlaps, average_depth, and partition_depth_histogram.[¹⁹][20]	
3	B	Materialized Views are automatically maintained by Snowflake in the background using serverless credits when source data changes.[²²][21]	
4	B	A column with a Dynamic Data Masking (Column-Level Security) policy on the source table prevents creating a Materialized View.[²²]	
5	C	SUBSTRING search type accelerates LIKE, CONTAINS, STARTSWITH on text columns — exactly what LIKE '%@gmail.com' needs.[¹⁵][23]	
6	C	Depth = 1 means perfect clustering: each distinct key value exists in exactly 1 micro-partition — maximum pruning efficiency.[¹⁹][17]	
7	B	Automatic Clustering is a background serverless process that continuously re-sorts micro-partitions to maintain the clustering key over time.[¹⁸][20]	
8	B	Result Cache is account-wide and shared — User B (same permissions, same query, no data change) gets the cached result immediately.[²⁴][26][²⁷]	

Q	Answer	Key Reason	
9	B, C, E	Wrapping clustered columns in functions (TO_CHAR, YEAR, DATE_TRUNC) prevents Snowflake from using partition metadata for pruning. Direct comparisons preserve pruning. ^{[4][16]}	
10	A, B, C, E	Pre-aggregation ✓; different clustering key on same data ✓; flatten VARIANT once ✓; speed up frequent subset queries ✓. MVs are NOT for raw data backups ✗. ^{[15][22]}	
11	B	ANYDML on the table invalidates all related Result Cache entries — Snowflake doesn't track which rows were changed and which cache entries are affected. ^{[24][26]}	
12	B	Clustering key is on region — it helps queries filtering on region. A filter on order_date gets no pruning benefit because order_date is not the clustering column. ^[^16]	
13	B	USE_CACHED_RESULT = FALSE bypasses the Result Cache — Snowflake always re-executes the query. Warehouse Local Cache is still active. ^[^26]	
14	C	Result Cache can hold a result for up to 31 days maximum, if the same query is re-run within each 24-hour window, extending the cache each time. ^[^24]	
15	C	SYSTEM\$ESTIMATE_AUTOMATIC_CLUSTERING_COSTS estimates the ongoing Automatic Clustering credit cost BEFORE enabling it. ^[^4]	
16	A	Clustering Keys improve range scans and ordered queries on the clustered column. SOS improves point lookups (equality), substring searches, and GEO queries. Different problem types. ^{[15][13]}	
17	A, B	Storage cost (pre-computed data stored) ✓; serverless compute for refresh ✓. Refresh does NOT use a virtual warehouse ✗; no extra clustering storage beyond the MV itself ✗; querying MV doesn't require warehouse (uses cached results) ✗. ^{[22][21]}	
18	B	Snowflake uses columnar storage — each column is stored separately. Selecting fewer columns reads fewer column files from micro-partitions → less I/O. ^[^28]	
19	B	Very high clustering depth = each value spans many micro-partitions = poor physical organization = most queries do full table scans regardless of filters. ^{[19][17]}	
20	B	Automatic Query Rewrite — Snowflake's query optimizer detects that a Materialized View can serve the query and transparently uses the MV instead of the base table. ^{[15][22]}	"]

References

1. [Free Practice Questions for Snowflake COF-CO2 Certification](#) - Subdomain 2.1: Outline security principles. · Subdomain 2.2: Define the entities and roles that are ...
2. [SnowPro Core Exam Prep 2026 - App Store - Apple](#) - Pass the Snowflake SnowPro Core exam with confidence. 2,100+ questions, progressive difficulty, and ...
3. [Monitor query activity with Query History - Snowflake Documentation](#) - The Query History page lets you explore queries executed in your Snowflake account over the last 14 ...
4. [Stop Guessing: The Holy Trinity of Snowflake Query Tuning](#) - Master Snowflake query tuning with this 5-step playbook: find bottlenecks, prune reads, reduce work,...
5. [Why Your Snowflake Joins Are Slow: Fix OR Joins Fast - Greybeam](#) - An OR in a Snowflake JOIN can trigger a Cartesian product and turn a 2-second query into minutes. Le...

6. [Snowflake performance tuning: Top 5 tips and techniques \(2026\)](#) - Disk spilling significantly affects query performance ...
7. [Problematic Query Patterns in Snowflake: Join Explosion](#) - A join explosion is only a problem if that join or a filter after it are where our performance probl...
8. [Creating and Implementing Dedicated Virtual Warehouse in Snowflake](#)
[Scaling Up for Performance](#) sub-headings 27 - Creating and Implementing a Dedicated Virtual Warehouse in Snowflake | Scaling Up for Performance | ...
9. [Warehouse considerations | Snowflake Documentation](#) - This topic provides general guidelines and best practices for using virtual warehouses in Snowflake ...
10. [Automatic Concurrency Scaling in Snowflake](#) - In auto-scale mode, Snowflake automatically adds or resumes additional clusters (up to the maximum n...
11. [Multi-cluster warehouses | Snowflake Documentation](#) - Multi-cluster warehouses enable you to scale compute resources to manage your user and query concurr...
12. [ALTER WAREHOUSE - Snowflake Documentation](#) - Object parameter that specifies the policy for automatically starting and shutting down clusters in ...
13. [Optimizing query performance | Snowflake Documentation](#) - You can optimize Snowflake query performance in the following ways: Search ... You can also use mate...
14. [Using the Query Acceleration Service \(QAS\)](#) - Snowflake only marks a query as eligible if there is high confidence that the query would be acceler...
15. [Optimizing query performance - Snowflake Documentation](#)
16. [Clustering Keys & Clustered Tables - Snowflake Documentation](#) - Use the system function, SYSTEM\$CLUSTERING_INFORMATION, to calculate clustering details, including c...
17. [Snowflake clustering 101: A beginner's guide \(2026\)](#) - Flexera - Snowflake Clustering keys are designated columns or ...
18. [Automatic Clustering As A...](#)
19. [SYSTEM\\$CLUSTERING_INFORMATION - Snowflake Documentation](#) - Most of the micro-partitions are grouped at the lower-end of the histogram, with the majority of mic...
20. [How to Monitor Clustering and Interpret Clustering Information in ...](#) - The system function SYSTEM\$CLUSTERING_DEPTH can provide an overview of how well a table is clustered...
21. [Enhancing Query Performance with Snowflake Materialized Views](#) - Snowflake Materialized Views offer a powerful solution to optimize query performance, reduce costs, ...
22. [Working with Materialized Views - Snowflake Documentation](#) - Regular views do not cache data, and therefore cannot improve performance by caching. However, in so...
23. [Search optimization service - Snowflake Documentation](#) - To inquire about upgrading, please contact Snowflake Support. The search optimization service can si...
24. [Snowflake Caching](#) - I have read in a few places that there are 3 levels of caching in Snowflake: Metadata cache. Maintai...
25. [Maximize Query Performance with the Results & Data Cache](#) - Snowflake uses a 3-layer caching model: Caches operate at the Cloud Services layer (Results and Meta...

26. Caching in Snowflake - Learn how Snowflake caching helps in improving the query performance by making data retrieval faster...
27. A deep dive into Snowflake's Caching Mechanism - Let's learn about how Snowflake keeps your queries running smoothly!
28. Snowflake Query Optimization: 16 tips to make your queries run faster - In this guide, we share 16 proven optimization techniques to maximize the performance and efficiency...
29. Understanding Snowflake Caching: Result Cache, Local Disk ... - Snowflake's Result Cache, Local Disk Cache, and Metadata Cache each play a crucial role in improving...
30. WAREHOUSE_METERING_HISTORY view¶
31. WAREHOUSE_METERING_HIS... - This Account Usage view can be used to return the hourly credit usage for a single warehouse (or all...